

第54章 DCMI—OV5640 摄像头

本章参考资料：《STM32F4xx 参考手册》、《STM32F4xx 规格书》、库帮助文档《stm32f4xx_dsp_stdperiph_lib_um.chm》。

关于开发板配套的 OV5640 摄像头参数可查阅《ov5640datasheet》配套资料获知。

STM32F4 芯片具有浮点运算单元，适合对图像信息使用 DSP 进行基本的图像处理，其处理速度比传统的 8、16 位机快得多，而且它还具有与摄像头通讯的专用 DCMI 接口，所以使用它驱动摄像头采集图像信息并进行基本的加工处理非常适合。本章讲解如何使用 STM32 驱动 OV5640 型号的摄像头。

54.1 摄像头简介

在各类信息中，图像含有最丰富的信息，作为机器视觉领域的核心部件，摄像头被广泛地应用在安防、探险以及车牌检测等场合。摄像头按输出信号的类型来看可以分为数字摄像头和模拟摄像头，按照摄像头图像传感器材料构成来看可以分为 CCD 和 CMOS。现在智能手机的摄像头绝大部分都是 CMOS 类型的数字摄像头。

54.1.1 数字摄像头跟模拟摄像头区别

❑ 输出信号类型

数字摄像头输出信号为数字信号，模拟摄像头输出信号为标准的模拟信号。

❑ 接口类型

数字摄像头有 USB 接口(比如常见的 PC 端免驱摄像头)、IEEE1394 火线接口(由苹果公司领导的开发联盟开发的一种高速度传送接口，数据传输率高达 800Mbps)、千兆网接口（网络摄像头）。模拟摄像头多采用 AV 视频端子（信号线+地线）或 S-VIDEO（即莲花头--SUPER VIDEO，是一种五芯的接口，由两路视频亮度信号、两路视频色度信号和一路公共屏蔽地线共五条芯线组成）。

❑ 分辨率

模拟摄像头的感光器件，其像素指标一般维持在 752(H)*582(V)左右的水平，像素数一般情况下维持在 41 万左右。现在的数字摄像头分辨率一般从数十万到数千万。但这并不能说明数字摄像头的成像分辨率就比模拟摄像头的高，原因在于模拟摄像头输出的是模拟视频信号，一般直接输入至电视或监视器，其感光器件的分辨率与电视信号的扫描数呈一定的换算关系，图像的显示介质已经确定，因此模拟摄像头的感光器件分辨率不是不能做高，而是依据于实际情况没必要做这么高。

54.1.2 CCD 与 CMOS 的区别

摄像头的图像传感器 CCD 与 CMOS 传感器主要区别如下：

❑ 成像材料

CCD 与 CMOS 的名称跟它们成像使用的材料有关, CCD 是“电荷耦合器件”(Charge Coupled Device)的简称, 而 CMOS 是“互补金属氧化物半导体”(Complementary Metal Oxide Semiconductor)的简称。

❑ 功耗

由于 CCD 的像素由 MOS 电容构成, 读取电荷信号时需使用电压相当大(至少 12V)的二相或三相或四相时序脉冲信号, 才能有效地传输电荷。因此 CCD 的取像系统除了要有多个电源外, 其外设电路也会消耗相当大的功率。有的 CCD 取像系统需消耗 2~5W 的功率。而 CMOS 光电传感器只需使用一个单电源 5V 或 3V, 耗电量非常小, 仅为 CCD 的 1/8~1/10, 有的 CMOS 取像系统只消耗 20~50mW 的功率。

❑ 成像质量

CCD 传感器制作技术起步早, 技术成熟, 采用 PN 结或二氧化硅(sio₂)隔离层隔离噪声, 所以噪声低, 成像质量好。与 CCD 相比, CMOS 的主要缺点是噪声高及灵敏度低, 不过现在随着 CMOS 电路降噪技术的不断发展, 为生产高密度优质的 CMOS 传感器提供了良好的条件, 现在的 CMOS 传感器已经占领了大部分的市场, 主流的单反相机、智能手机都已普遍采用 CMOS 传感器。

54.2 OV5640 摄像头

本章主要讲解实验板配套的摄像头, 它的实物见图 54-1, 该摄像头主要由镜头、图像传感器、板载电路及下方的信号引脚组成。

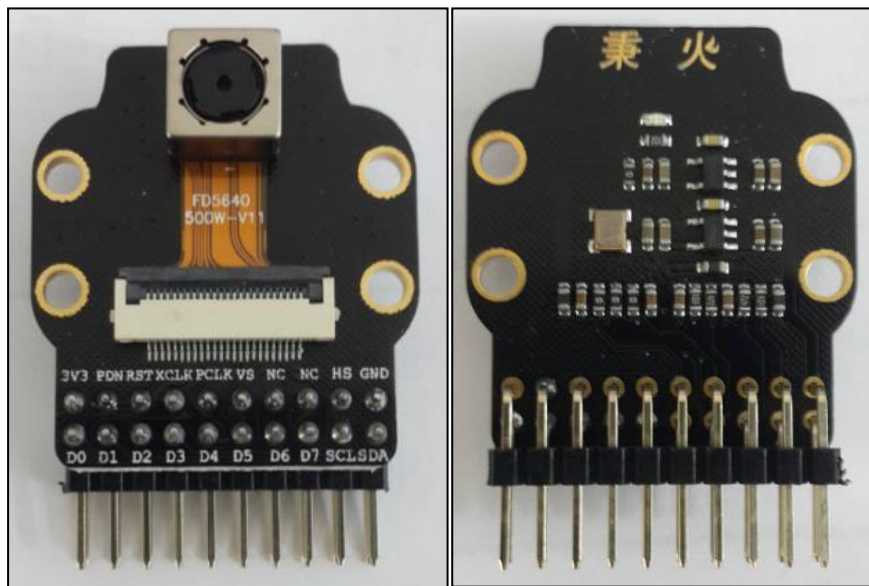


图 54-1 实验板配套的 OV5640 摄像头

镜头部件包含一个镜头座和一个可旋转调节距离的凸透镜, 通过旋转可以调节焦距, 正常使用时, 镜头座覆盖在电路板上遮光, 光线只能经过镜头传输到正中央的图像传感器, 它采集光线信号, 然后把采集得的数据通过下方的信号引脚输出数据到外部器件。

54.2.1 OV5640 传感器简介

图像传感器是摄像头的核心部件，上述摄像头中的图像传感器是一款型号为 OV5640 的 CMOS 类型数字图像传感器。该传感器支持输出最大为 500 万像素的图像 (2592x1944 分辨率)，支持使用 VGA 时序输出图像数据，输出图像的数据格式支持 YUV(422/420)、YCbCr422、RGB565 以及 JPEG 格式，若直接输出 JPEG 格式的图像时可大大减少数据量，方便网络传输。它还可以对采集得的图像进行补偿，支持伽玛曲线、白平衡、饱和度、色度等基础处理。根据不同的分辨率配置，传感器输出图像数据的帧率从 15-60 帧可调，工作时功率在 150mW-200mW 之间。

54.2.2 OV5640 引脚及功能框图

OV5640 模组带有自动对焦功能，引脚的定义见图 54-2。

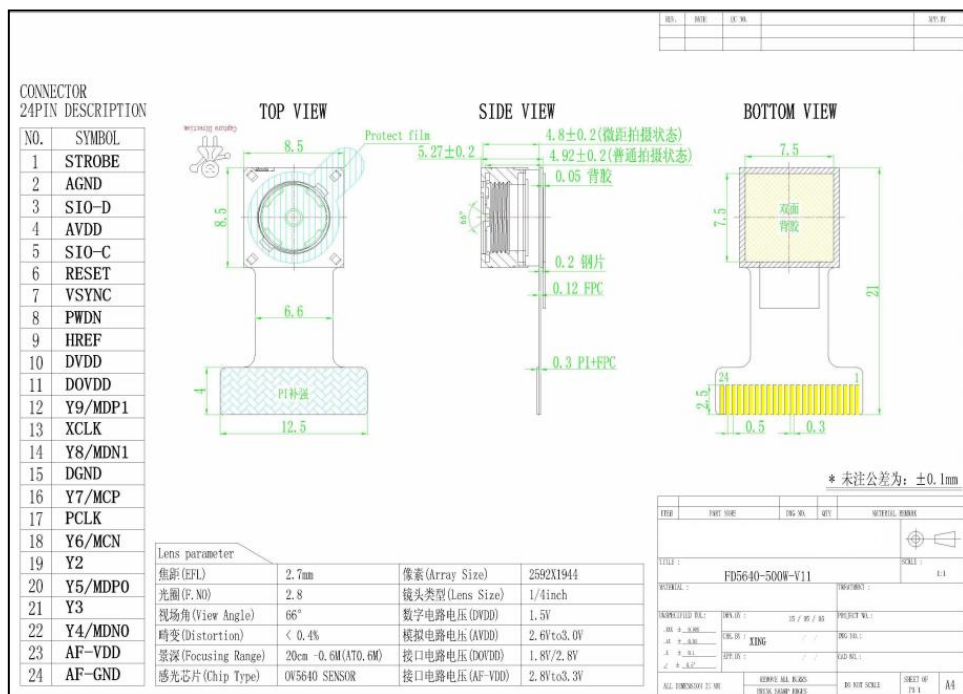


图 54-2 OV5640 传感器引脚分布图

信号引脚功能介绍如下，介绍如下表 54-1。

表 54-1 OV5640 管脚

管脚名称	管脚类型	管脚描述
SIO_C	输入	SCCB 总线的时钟线，可类比 I2C 的 SCL
SIO_D	I/O	SCCB 总线的数据线，可类比 I2C 的 SDA
RESET	输入	系统复位管脚，低电平有效
PWDN	输入	掉电/省电模式，高电平有效
HREF	输出	行同步信号
VSYNC	输出	帧同步信号
PCLK	输出	像素同步时钟输出信号
XCLK	输入	外部时钟输入端口，可接外部晶振
Y2...Y9	输出	像素数据输出端口

下面我们配合图 54-3 中的 OV5640 功能框图讲解这些信号引脚。

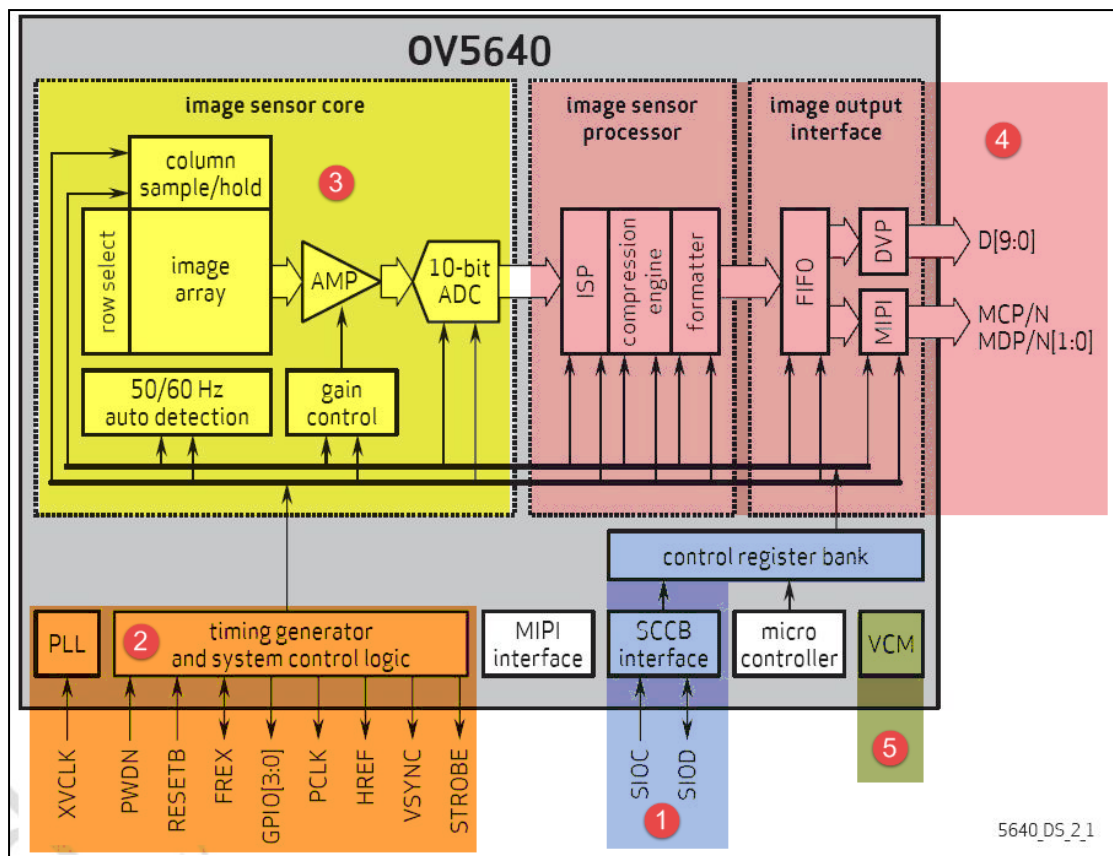


图 54-3 OV5640 功能框图

(1) 控制寄存器

标号①处的是 OV5640 的控制寄存器，它根据这些寄存器配置的参数来运行，而这些参数是由外部控制器通过 SIO_C 和 SIO_D 引脚写入的，SIO_C 与 SIO_D 使用的通讯协议跟 I2C 十分类似，在 STM32 中我们完全可以直接用 I2C 硬件外设来控制。

(2) 通信、控制信号及时钟

标号②处包含了 OV5640 的通信、控制信号及外部时钟，其中 PCLK、HREF 及 VSYNC 分别是像素同步时钟、行同步信号以及帧同步信号，这与液晶屏控制中的信号是很类似的。RESETB 引脚为低电平时，用于复位整个传感器芯片，PWDN 用于控制芯片进入低功耗模式。注意最后的一个 XCLK 引脚，它跟 PCLK 是完全不同的，XCLK 是用于驱动整个传感器芯片的时钟信号，是外部输入到 OV5640 的信号；而 PCLK 是 OV5640 输出数据时的同步信号，它是由 OV5640 输出的信号。XCLK 可以外接晶振或由外部控制器提供，若要类比 XCLK 之于 OV5640 就相当于 HSE 时钟输入引脚与 STM32 芯片的关系，PCLK 引脚可类比 STM32 的 I2C 外设的 SCL 引脚。

(3) 感光矩阵

标号③处的是感光矩阵，光信号在这里转化成电信号，经过各种处理，这些信号存储成由一个个像素点表示的数字图像。

(4) 数据输出信号

标号④处包含了 DSP 处理单元，它会根据控制寄存器的配置做一些基本的图像处理运算。这部分还包含了图像格式转换单元及压缩单元，转换出的数据最终通过 Y0-Y9 引脚输出，一般来说我们使用 8 根数据线来传输，这时仅使用 Y2-Y9 引脚，OV5640 与外部器件的连接方式见图 54-4。

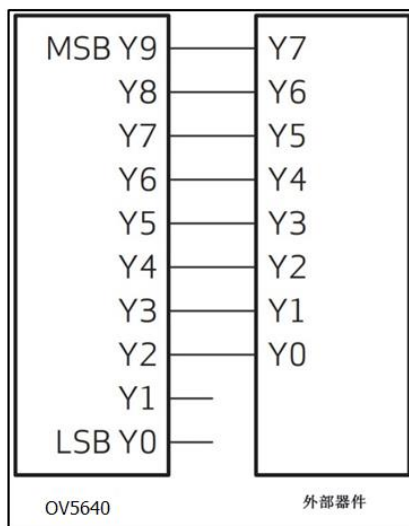


图 54-4 8 位数据线接法

(5) 数据输出信号

标号⑤处为 VCM 处理单元，他会通过图像分析来实现图像的自动对焦功能。要实现自动对焦还需要下载自动对焦固件到模组，后面摄像头实验详细介绍这个功能。

54.2.3 SCCB 时序

外部控制器对 OV5640 寄存器的配置参数是通过 SCCB 总线传输过去的，而 SCCB 总线跟 I2C 十分类似，所以在 STM32 驱动中我们直接使用片上 I2C 外设与它通讯。SCCB 与标准的 I2C 协议的区别是它每次传输只能写入或读取一个字节的数据，而 I2C 协议是支持突发读写的，即在一次传输中可以写入多个字节的数据(EEPROM 中的页写入时序即突发写)。关于 SCCB 协议的完整内容可查看配套资料里的《SCCB 协议》文档，下面我们简单介绍下。

SCCB 的起始、停止信号及数据有效性

SCCB 的起始信号、停止信号及数据有效性与 I2C 完全一样，见图 54-5 及图 54-6。

- ❑ 起始信号：在 SIO_C 为高电平时，SIO_D 出现一个下降沿，则 SCCB 开始传输。
- ❑ 停止信号：在 SIO_C 为高电平时，SIO_D 出现一个上升沿，则 SCCB 停止传输。
- ❑ 数据有效性：除了开始和停止状态，在数据传输过程中，当 SIO_C 为高电平时，必须保证 SIO_D 上的数据稳定，也就是说，SIO_D 上的电平变换只能发生在 SIO_C 为低电平时的时候，SIO_D 的信号在 SIO_C 为高电平时被采集。

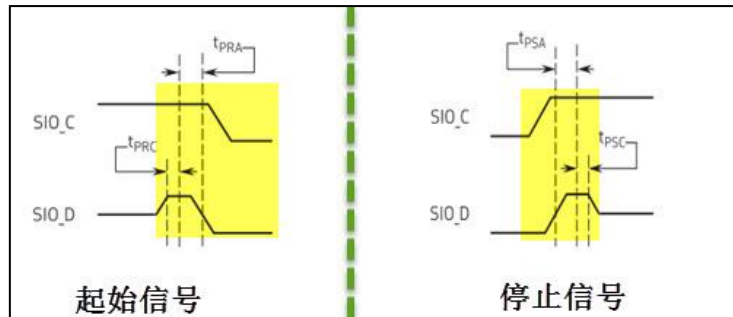


图 54-5 SCCB 停止信号

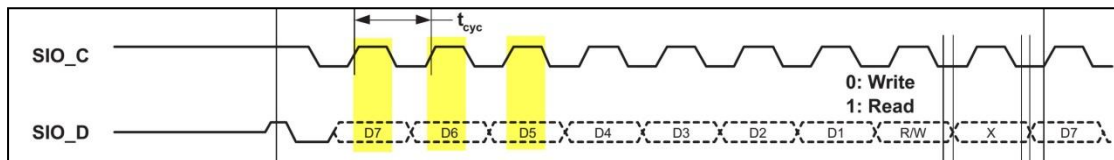


图 54-6 SCCB 的数据有效性

SCCB 数据读写过程

在 SCCB 协议中定义的读写操作与 I2C 也是一样的，只是换了一种说法。它定义了两种写操作，即三步写操作和两步写操作。三步写操作可向从设备的一个目的寄存器中写入数据，见图 54-7。在三步写操作中，第一阶段发送从设备的 ID 地址+W 标志(等于 I2C 的设备地址：7 位设备地址+读写方向标志)，第二阶段发送从设备目标寄存器的 16 位地址，第三阶段发送要写入寄存器的 8 位数据。图中的“X”数据位可写入 1 或 0，对通讯无影响。

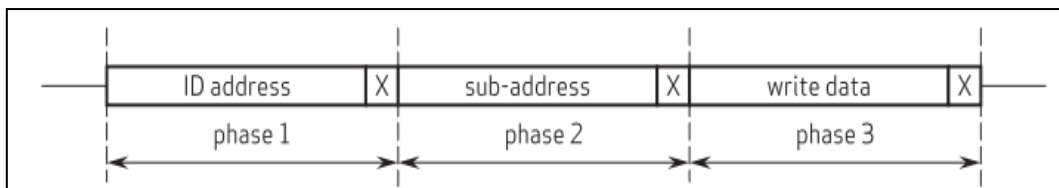


图 54-7 SCCB 的三步写操作

而两步写操作没有第三阶段，即只向从器件传输了设备 ID+W 标志和目的寄存器的地址，见图 54-8。两步写操作是用来配合后面的读寄存器数据操作的，它与读操作一起使用，实现 I2C 的复合过程。

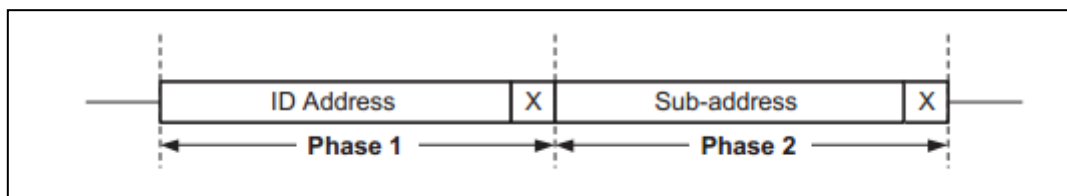


图 54-8 SCCB 的两步写操作

两步读操作，它用于读取从设备目的寄存器中的数据，见图 54-9。在第一阶段中发送从设备的设备 ID+R 标志(设备地址+读方向标志)和自由位，在第二阶段中读取寄存器中的 8 位数据和写 NA 位(非应答信号)。由于两步读操作没有确定目的寄存器的地址，所以在读操作前，必需有一个两步写操作，以提供读操作中的寄存器地址。

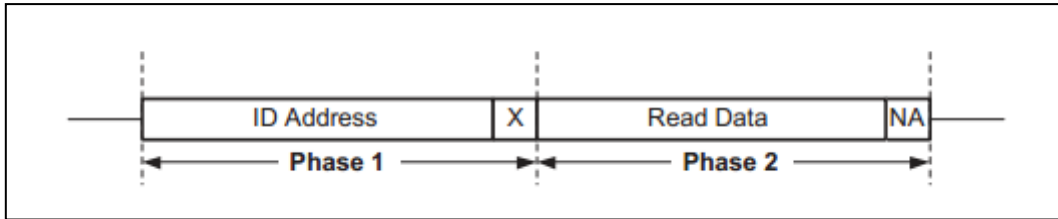


图 54-9 SCCB 的两步读操作

可以看到，以上介绍的 SCCB 特性都与 I2C 无区别，而 I2C 比 SCCB 还多出了突发读写功能，所以 SCCB 可以看作是 I2C 的子集，我们完全可以使用 STM32 的 I2C 外设来与 OV5640 进行 SCCB 通讯。

54.2.4 OV5640 的寄存器

控制 OV5640 涉及到它很多的寄存器，可直接查询《ov5640datasheet》了解，通过这些寄存器的配置，可以控制它输出图像的分辨率大小、图像格式及图像方向等。要注意的是 OV5640 寄存器地址为 16 位。

官方还提供了一个《OV5640_自动对焦照相模组应用指南(DVP_接口)_R2.13C.pdf》的文档，它针对不同的配置需求，提供了配置范例，见图 54-10。其中 write_SCCB 是一个利用 SCCB 向寄存器写入数据的函数，第一个参数为要写入的寄存器的地址，第二个参数为要写入的内容。

4.1.2 VGA 预览

VGA 30 帧/秒

```
// YUV VGA 30fps, night mode 5fps
// Input Clock = 24Mhz, PCLK = 56MHz
write_i2c(0x3035, 0x11);    // PLL
write_i2c(0x3036, 0x46);    // PLL
write_i2c(0x3c07, 0x08);    // light meter 1 threshold [7:0]
write_i2c(0x3820, 0x41);    // Sensor flip off, ISP flip on
write_i2c(0x3821, 0x07);    // Sensor mirror on, ISP mirror on, H binning on
```

图 54-10 调节帧率的寄存器配置范例

54.2.5 像素数据输出时序

对 OV5640 采用 SCCB 协议进行控制，而它输出图像时则使用 VGA 时序(还可用 SVGA、UXGA，这些时序都差不多)，这跟控制液晶屏输入图像时很类似。OV5640 输出图像时，一帧帧地输出，在帧内的数据一般从左到右，从上到下，一个像素一个像素地输出(也可通过寄存器修改方向)，见图 54-11。

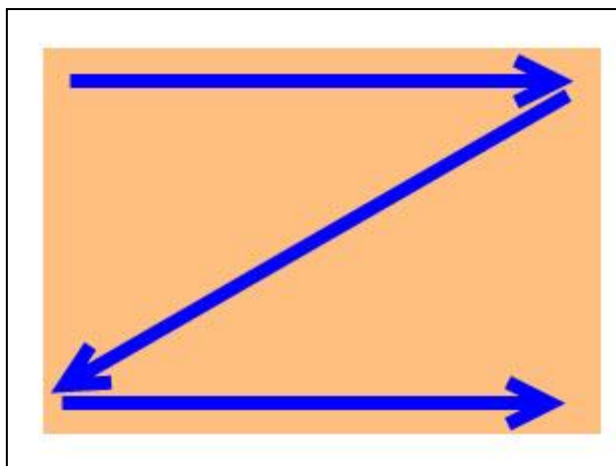


图 54-11 摄像头数据输出

例如，图 54-12，若我们使用 Y2-Y9 数据线，图像格式设置为 RGB565，进行数据输出时，Y2-Y9 数据线会在 1 个像素同步时钟 PCLK 的驱动下发送 1 字节的数据信号，所以 2 个 PCLK 时钟可发送 1 个 RGB565 格式的像素数据。像素数据依次传输，每传输完一行数据时，行同步信号 HREF 会输出一个电平跳变信号，每传输完一帧图像时，VSYNC 会输出一个电平跳变信号。

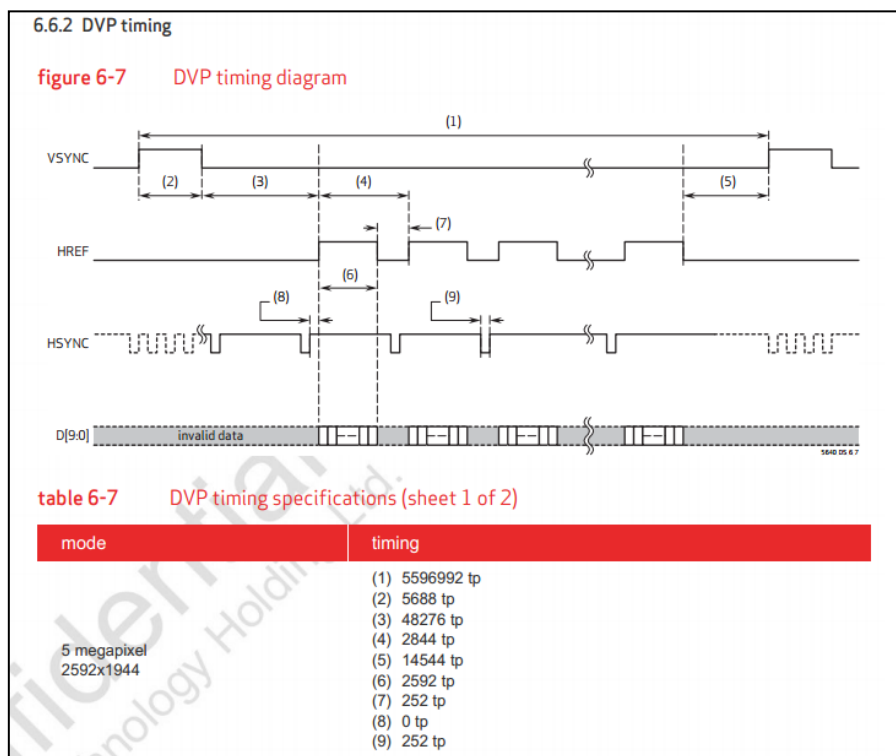


图 54-12 DVP 接口时序

54.3 STM32 的 DCMI 接口简介

STM32F4 系列的控制器包含了 DCMI 数字摄像头接口(Digital camera Interface)，它支持使用上述类似 VGA 的时序获取图像数据流，支持原始的按行、帧格式来组织的图像数

据，如 YUV、RGB，也支持接收 JPEG 格式压缩的数据流。接收数据时，主要使用 HSYNC 及 VSYNC 信号来同步。

54.3.1 DCMI 整体框图

STM32 的 DCMI 接口整体框图见图 54-13。

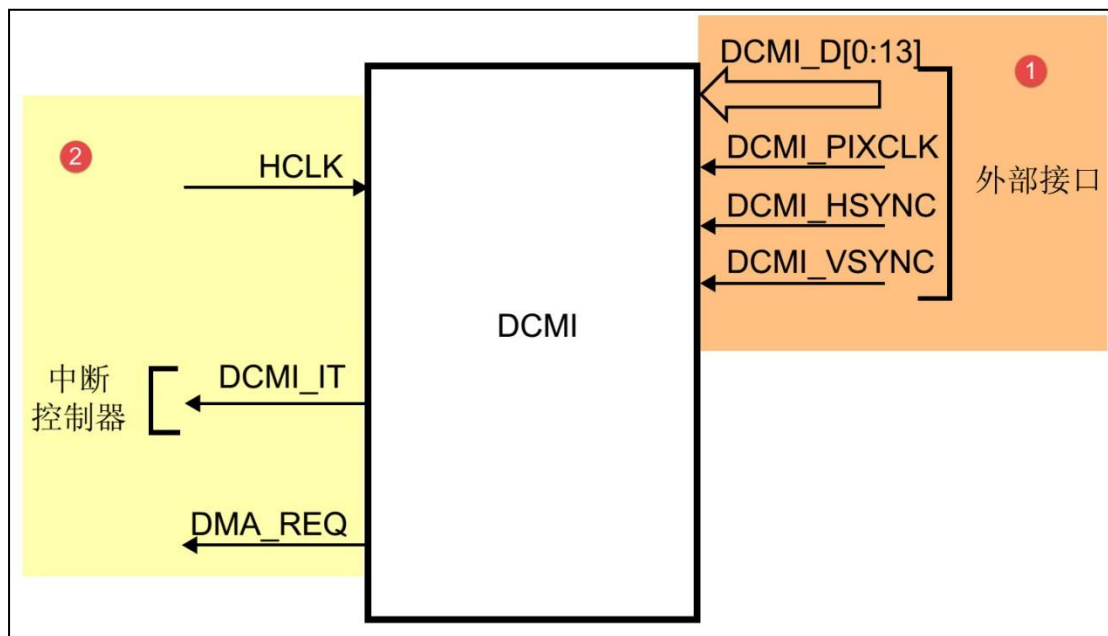


图 54-13 DCMI 接口整体框图

外部接口及时序

上图标号①处的是 DCMI 向外部引出的信号线。DCMI 提供的外部接口的方向都是输入的，接口的各个信号线说明见表 54-2。

表 54-2 DCMI 的信号线说明

引脚名称	说明
DCMI_D[0:13]	数据线
DCMI_PIXCLK	像素同步时钟
DCMI_HSYNC	行同步信号(水平同步信号)
DCMI_VSYNC	帧同步信号(垂直同步信号)

其中 DCMI_D 数据线的数量可选 8、10、12 或 14 位，各个同步信号的有效极性都可编程控制。它使用的通讯时序与 OV5640 的图像数据输出接口时序一致，见图 54-14。

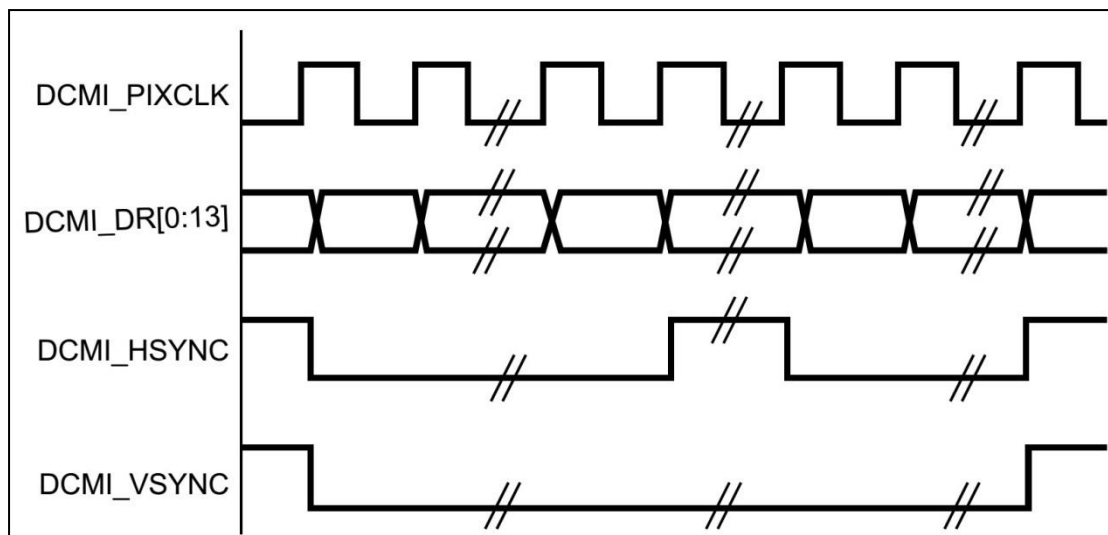


图 54-14 DCMI 时序图

内部信号及 PIXCLK 的时钟频率

图 54-13 的标号②处表示 DCMI 与内部的信号线。在 STM32 的内部，使用 HCLK 作为时钟源提供给 DCMI 外设。从 DCMI 引出有 DCMI_IT 信号至中断控制器，并可通过 DMA_REQ 信号发送 DMA 请求。

DCMI 从外部接收数据时，在 HCLK 的上升沿时对 PIXCLK 同步的信号进行采样，它限制了 PIXCLK 的最小时钟周期要大于 2.5 个 HCLK 时钟周期，即最高频率为 HCLK 的 1/4。

54.3.2 DCMI 接口内部结构

DCMI 接口的内部结构见图 54-15。

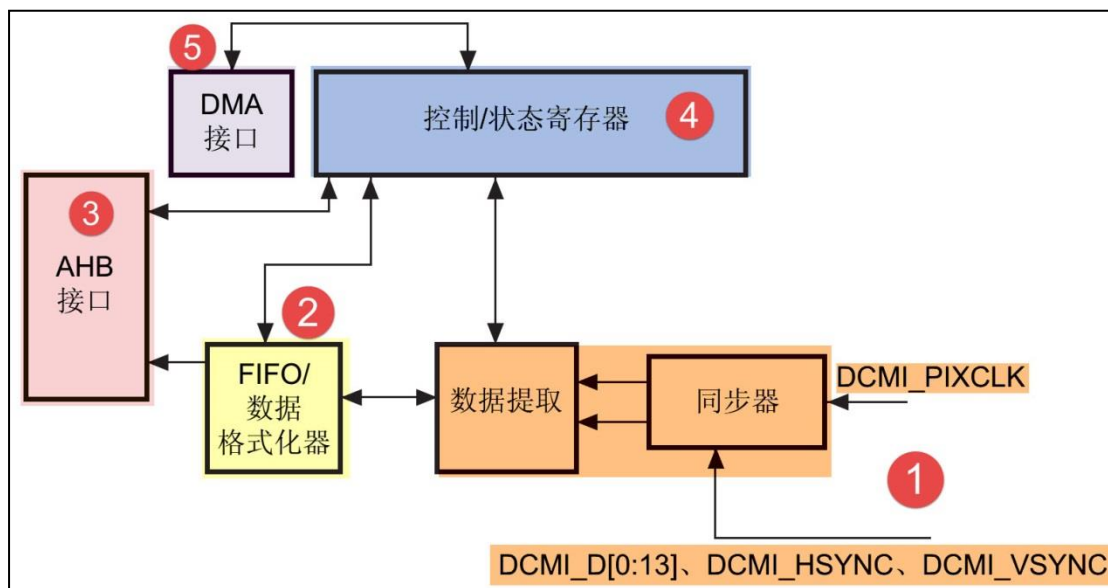


图 54-15 DCMI 接口内部结构

(1) 同步器

同步器主要用于管理 DCMI 接收数据的时序，它根据外部的信号提取输入的数据。

(2) FIFO/数据格式化器

为了对数据传输加以管理，STM32 在 DCMI 接口上实现了 4 个字(32bit x4)深度的 FIFO，用以缓冲接收到的数据。

(3) AHB 接口

DCMI 接口挂载在 AHB 总线上，在 AHB 总线中有一个 DCMI 接口的数据寄存器，当我们读取该寄存器时，它会从 FIFO 中获取数据，并且 FIFO 中的数据指针会自动进行偏移，使得我们每次读取该寄存器都可获得一个新的数据。

(4) 控制/状态寄存器

DCMI 的控制寄存器协调图中的各个结构运行，程序中可通过检测状态寄存器来获 DCMI 的当前运行状态。

(5) DMA 接口

由于 DCMI 采集的数据量很大，我们一般使用 DMA 来把采集得的数据搬运至内存。

54.3.3 同步方式

DCMI 接口支持硬件同步或内嵌码同步方式，硬件同步方式即使用 HSYNC 和 VSYNC 作为同步信号的方式，OV5640 就是使用这种同步时序。

而内嵌码同步的方式是使用数据信号线传输中的特定编码来表示同步信息，由于需要用 0x00 和 0xFF 来表示编码，所以表示图像的数据中不能包含有这两个值。利用这两个值，它扩展到 4 个字节，定义出了 2 种模式的同步码，每种模式包含 4 个编码，编码格式为 0xFF0000XY，其中 XY 的值可通过寄存器设置。当 DCMI 接收到这样的编码时，它不会把这些当成图像数据，而是按照表 54-3 中的编码来解释，作为同步信号。

表 54-3 两种模式的内嵌码

模式 2 的内嵌码	模式 1 的内嵌码
帧开始(FS)	有效行开始(SAV)
帧结束(FE)	有效行结束(EAV)
行开始(LS)	帧间消隐期内的行开始(SAV)，其中消隐期内的即为无效数据
行结束(LE)	帧间消隐期内的行结束(EAV)，其中消隐期内的即为无效数据

54.3.4 捕获模式及捕获率

DCMI 还支持两种数据捕获模式，分别为快照模式和连续采集模式。快照模式时只采集一帧的图像数据，连续采集模式会一直采集多个帧的数据，并且可以通过配置捕获率来控制采集多少数据，如可配置为采集所有数据或隔 1 帧采集一次数据或隔 3 帧采集一次数据。

54.4 DCMI 初始化结构体

与其它外设一样，STM32 的 DCMI 外设也可以使用库函数来控制，其中最主要的配置项都封装到了 DCMI_InitTypeDef 结构体，来这些内容都定义在库文件“stm32f4xx_dcmi.h”

及“stm32f4xx_dcmi.c”中，编程时我们可以结合这两个文件内的注释使用或参考库帮助文档。

DCMI_InitTypeDef 初始化结构体的内容见代码清单 54-1。

代码清单 54-1 DCMI 初始化结构体

```
1 /**
2  * @brief   DCMI 初始化结构体
3  */
4 typedef struct
5 {
6     uint16_t DCMI_CaptureMode;      /*选择连续模式或拍照模式 */
7     uint16_t DCMI_SynchroMode;      /*选择硬件同步模式还是内嵌码模式 */
8     uint16_t DCMI_PCKPolarity;      /*设置像素时钟的有效边沿*/
9     uint16_t DCMI_VSPolarity;       /*设置 VSYNC 的有效电平*/
10    uint16_t DCMI_HSPolarity;        /*设置 HSYNC 的有效边沿*/
11    uint16_t DCMI_CaptureRate;       /*设置图像的采集间隔 */
12    uint16_t DCMI_ExtendedDataMode; /*设置数据线的宽度 */
13 } DCMI_InitTypeDef;
```

这些结构体成员说明如下，其中括号内的文字是对应参数在 STM32 标准库中定义的宏：

(1) DCMI_CaptureMode

本成员设置 DCMI 的捕获模式，可以选择为连续摄像(DCMI_CaptureMode_Continuous)或单张拍照 DCMI_CaptureMode_SnapShot。

(2) DCMI_SynchroMode

本成员设置 DCMI 数据的同步模式，可以选择为硬件同步方式(DCMI_SynchroMode_Hardware)或内嵌码方式(DCMI_SynchroMode_Embedded)。

(3) DCMI_PCKPolarity

本成员用于配置 DCMI 接口像素时钟的有效边沿，即在该时钟边沿时，DCMI 会对数据线上的信号进行采样，它可以被设置为上升沿有效(DCMI_PCKPolarity_Rising)或下降沿有效(DCMI_PCKPolarity_Falling)。

(4) DCMI_VSPolarity

本成员用于设置 VSYNC 的有效电平，当 VSYNC 信号线表示为有效电平时，表示新的一帧数据传输完成，它可以被设置为高电平有效(DCMI_VSPolarity_High)或低电平有效(DCMI_VSPolarity_Low)。

(5) DCMI_HSPolarity

类似地，本成员用于设置 HSYNC 的有效电平，当 HSYNC 信号线表示为有效电平时，表示新的一行数据传输完成，它可以被设置为高电平有效(DCMI_HSPolarity_High)或低电平有效(DCMI_HSPolarity_Low)。

(6) DCMI_CaptureRate

本成员可以用于设置 DCMI 捕获数据的频率，可以设置为全采集、半采集或 1/4 采集(DCMI_CaptureRate_All_Frame/ 1of2_Frame/ 1of4_Frame)，在间隔采集的情况下，STM32 的 DCMI 外设会直接按间隔丢弃数据。

(7) DCMI_ExtendedDataMode

本成员用于设置 DCMI 的数据线宽度，可配置为 8/10/12 及 14 位数据线宽(DCMI_ExtendedDataMode_8b/10b/12b/14b)。

配置完这些结构体成员后，我们调用库函数 DCMI_Init 即可把这些参数写入到 DCMI 的控制寄存器中，实现 DCMI 的初始化。

54.5 DCMI—OV5640 摄像头实验

本小节讲解如何使用 DCMI 接口从 OV5640 摄像头输出的 RGB565 格式的图像数据，并把这些数据实时显示到液晶屏上。

学习本小节内容时，请打开配套的“DCMI—OV5640 摄像头”工程配合阅读。

54.5.1 硬件设计

摄像头原理图

本实验采用的 OV5640 摄像头实物见图 54-16，其原理图见图 54-17。

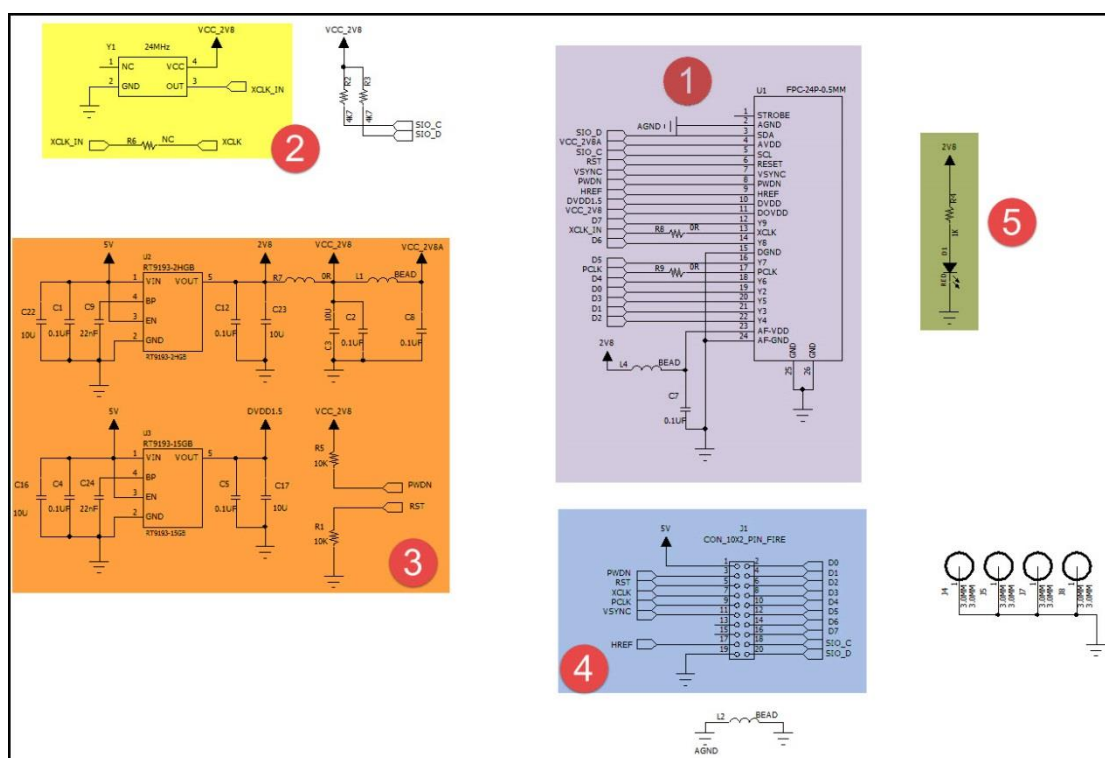


图 54-16 OV5640 摄像头原理图

图 54-16 标号①处的是 OV5640 模组接口电路，在这部分中已对 SCCB 使用的信号线接了上拉电阻，外部电路可以省略上拉；标号②处的是一个 24MHz 的有源晶振，它为 OV5640 提供系统时钟，如果不想使用外部晶振提供时钟源，可以参考图中的 R6 处贴上 0 欧电阻，XCLK 引脚引出至外部，由外部控制器提供时钟；标号③处的是电源转换模块，可以从 5V 转 2.8V 和 1.5V 供给模组使用；标号④处的是摄像头引脚集中引出的排针接口，使用它可以方便地与 STM32 实验板中的排母连接。标号⑤处的是电源指示灯。

摄像头与实验板的连接

通过排母，OV5640 与 STM32 引脚的连接关系见图 54-17。

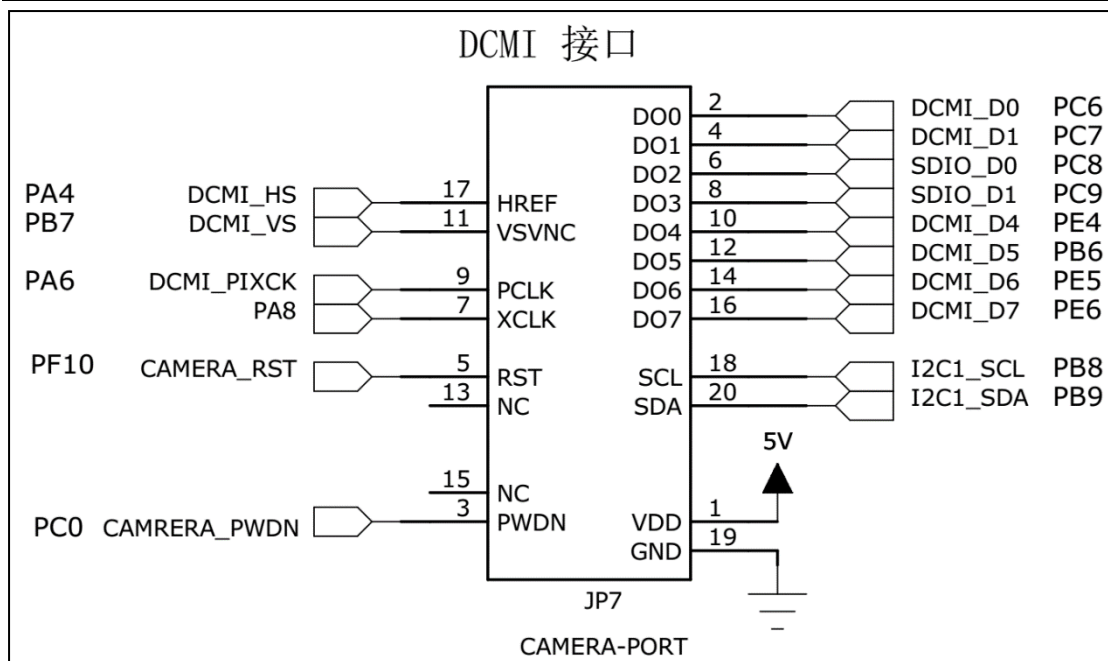


图 54-17 STM32 实验板引出的 DCMI 接口

以上原理图可查阅《ov5640—黑白原理图》及《野火 F407 开发板底板原理图》文档获知，若您使用的摄像头或实验板不一样，请根据实际连接的引脚修改程序。

54.5.2 软件设计

为了使工程更加有条理，我们把摄像头控制相关的代码独立分开存储，方便以后移植。在“液晶显示”工程的基础上新建“bsp_ov5640.c”，“ov5640_AF.c”，“bsp_ov5640.h”，“ov5640_AF.h”文件，这些文件也可根据您的喜好命名，它们不属于 STM32 标准库的内容，是由我们自己根据应用需要编写的。

1. 编程要点

- (1) 初始化 DCMI 时钟，I2C 时钟；
- (2) 使用 I2C 接口向 OV5640 写入寄存器配置；
- (3) 初始化 DCMI 工作模式；
- (4) 初始化 DMA，用于搬运 DCMI 的数据到显存空间进行显示；
- (5) 编写测试程序，控制采集图像数据并显示到液晶屏。

2. 代码分析

摄像头硬件相关宏定义

我们把摄像头控制硬件相关的配置都以宏的形式定义到“bsp_ov5640.h”文件中，其中包括 I2C 及 DCMI 接口的，见代码清单 54-2。

代码清单 54-2 摄像头硬件配置相关的宏(省略了部分数据线)

```
1
2 /*摄像头接口 */
```

```

3 //IIC SCCB
4 #define CAMERA_I2C                                I2C1
5 #define CAMERA_I2C_CLK                            RCC_APB1Periph_I2C1
6
7 #define CAMERA_I2C_SCL_PIN                        GPIO_Pin_8
8 #define CAMERA_I2C_SCL_GPIO_PORT                 GPIOB
9 #define CAMERA_I2C_SCL_GPIO_CLK                  RCC_AHB1Periph_GPIOB
10 #define CAMERA_I2C_SCL_SOURCE                    GPIO_PinSource8
11 #define CAMERA_I2C_SCL_AF                        GPIO_AF_I2C1
12
13 #define CAMERA_I2C_SDA_PIN                        GPIO_Pin_9
14 #define CAMERA_I2C_SDA_GPIO_PORT                 GPIOB
15 #define CAMERA_I2C_SDA_GPIO_CLK                  RCC_AHB1Periph_GPIOB
16 #define CAMERA_I2C_SDA_SOURCE                    GPIO_PinSource9
17 #define CAMERA_I2C_SDA_AF                        GPIO_AF_I2C1
18
19 //VSYNC
20 #define DCMI_VSYNC_GPIO_PORT                     GPIOB
21 #define DCMI_VSYNC_GPIO_CLK                      RCC_AHB1Periph_GPIOB
22 #define DCMI_VSYNC_GPIO_PIN                     GPIO_Pin_7
23 #define DCMI_VSYNC_PINSOURCE                    GPIO_PinSource7
24 #define DCMI_VSYNC_AF                            GPIO_AF_DCMI
25 // HSYNC
26 #define DCMI_HSYNC_GPIO_PORT                     GPIOA
27 #define DCMI_HSYNC_GPIO_CLK                      RCC_AHB1Periph_GPIOA
28 #define DCMI_HSYNC_GPIO_PIN                     GPIO_Pin_4
29 #define DCMI_HSYNC_PINSOURCE                    GPIO_PinSource4
30 #define DCMI_HSYNC_AF                            GPIO_AF_DCMI
31 //PIXCLK
32 #define DCMI_PIXCLK_GPIO_PORT                     GPIOA
33 #define DCMI_PIXCLK_GPIO_CLK                      RCC_AHB1Periph_GPIOA
34 #define DCMI_PIXCLK_GPIO_PIN                     GPIO_Pin_6
35 #define DCMI_PIXCLK_PINSOURCE                    GPIO_PinSource6
36 #define DCMI_PIXCLK_AF                            GPIO_AF_DCMI
37 //PWDN
38 #define DCMI_PWDN_GPIO_PORT                       GPIOG
39 #define DCMI_PWDN_GPIO_CLK                       RCC_AHB1Periph_GPIOG
40 #define DCMI_PWDN_GPIO_PIN                       GPIO_Pin_3
41
42 //数据信号线
43 #define DCMI_D0_GPIO_PORT                         GPIOC
44 #define DCMI_D0_GPIO_CLK                         RCC_AHB1Periph_GPIOC
45 #define DCMI_D0_GPIO_PIN                         GPIO_Pin_0
46 #define DCMI_D0_PINSOURCE                        GPIO_PinSource0
47 #define DCMI_D0_AF                                GPIO_AF_DCMI
48 /*.....省略部分数据线*/

```

以上代码根据硬件的连接，把与 DCMI、I2C 接口与摄像头通讯使用的引脚号、引脚源以及复用功能映射都以宏封装起来。

初始化 DCMI 的 GPIO 及 I2C

利用上面的宏，初始化 DCMI 的 GPIO 引脚及 I2C，见代码清单 54-3。

代码清单 54-3 初始化 DCMI 的 GPIO 及 I2C(省略了部分数据线)

```

1 /**
2  * @brief  初始化控制摄像头使用的 GPIO(I2C/DCMI)
3  * @param  None
4  * @retval None
5  */
6 void OV5640_HW_Init(void)
7 {
8     GPIO_InitTypeDef GPIO_InitStructure;
9     I2C_InitTypeDef I2C_InitStructure;
10
11     /***DCMI 引脚配置***/

```

```
12  /* 使能 DCMI 时钟 */
13  RCC_AHB1PeriphClockCmd(DCMI_PWDN_GPIO_CLK|DCMI_RST_GPIO_CLK|DCMI_VS
14                          YNC_GPIO_CLK | DCMI_HSYNC_GPIO_CLK |
15                          DCMI_PIXCLK_GPIO_CLK|
16                          DCMI_D0_GPIO_CLK| DCMI_D1_GPIO_CLK|
17                          DCMI_D2_GPIO_CLK| DCMI_D3_GPIO_CLK|
18                          DCMI_D4_GPIO_CLK| DCMI_D5_GPIO_CLK|
19                          DCMI_D6_GPIO_CLK| DCMI_D7_GPIO_CLK, ENABLE);
20
21  /*控制/同步信号线*/
22  GPIO_InitStructure.GPIO_Pin = DCMI_VSYNC_GPIO_PIN;
23  GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
24  GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
25  GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
26  GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP ;
27  GPIO_Init(DCMI_VSYNC_GPIO_PORT, &GPIO_InitStructure);
28  GPIO_PinAFConfig(DCMI_VSYNC_GPIO_PORT, DCMI_VSYNC_PINSOURCE,
29                  DCMI_VSYNC_AF);
30
31  GPIO_InitStructure.GPIO_Pin = DCMI_HSYNC_GPIO_PIN ;
32  GPIO_Init(DCMI_HSYNC_GPIO_PORT, &GPIO_InitStructure);
33  GPIO_PinAFConfig(DCMI_HSYNC_GPIO_PORT, DCMI_HSYNC_PINSOURCE,
34                  DCMI_HSYNC_AF);
35
36  GPIO_InitStructure.GPIO_Pin = DCMI_PIXCLK_GPIO_PIN ;
37  GPIO_Init(DCMI_PIXCLK_GPIO_PORT, &GPIO_InitStructure);
38  GPIO_PinAFConfig(DCMI_PIXCLK_GPIO_PORT, DCMI_PIXCLK_PINSOURCE,
39                  DCMI_PIXCLK_AF);
40
41  /*数据信号*/
42  GPIO_InitStructure.GPIO_Pin = DCMI_D0_GPIO_PIN ;
43  GPIO_Init(DCMI_D0_GPIO_PORT, &GPIO_InitStructure);
44  GPIO_PinAFConfig(DCMI_D0_GPIO_PORT, DCMI_D0_PINSOURCE, DCMI_D0_AF);
45  /*...省略部分数据信号线*/
46
47  GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
48  GPIO_InitStructure.GPIO_Speed = GPIO_Speed_2MHz;
49  GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
50  GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
51  GPIO_InitStructure.GPIO_Pin = DCMI_PWDN_GPIO_PIN ;
52  GPIO_Init(DCMI_PWDN_GPIO_PORT, &GPIO_InitStructure);
53  GPIO_InitStructure.GPIO_Pin = DCMI_RST_GPIO_PIN ;
54  GPIO_Init(DCMI_RST_GPIO_PORT, &GPIO_InitStructure);
55  /*PWDN 引脚, 高电平关闭电源, 低电平供电*/
56
57  GPIO_ResetBits(DCMI_RST_GPIO_PORT,DCMI_RST_GPIO_PIN);
58  GPIO_SetBits(DCMI_PWDN_GPIO_PORT,DCMI_PWDN_GPIO_PIN);
59
60  Delay(10); //延时 10ms
61
62  GPIO_ResetBits(DCMI_PWDN_GPIO_PORT,DCMI_PWDN_GPIO_PIN);
63
64  Delay(10); //延时 10ms
65
66  GPIO_SetBits(DCMI_RST_GPIO_PORT,DCMI_RST_GPIO_PIN);
67
68  /***** 配置 I2C, 使用 I2C 与摄像头的 SCCB 接口通讯*****/
69  /* 使能 I2C 时钟 */
70  RCC_APB1PeriphClockCmd(CAMERA_I2C_CLK, ENABLE);
71  /* 使能 I2C 使用的 GPIO 时钟 */
72  RCC_AHB1PeriphClockCmd(CAMERA_I2C_SCL_GPIO_CLK|CAMERA_I2C_SDA_GPIO_
73                          CLK, ENABLE);
74  /* 配置引脚源 */
75  GPIO_PinAFConfig(CAMERA_I2C_SCL_GPIO_PORT, CAMERA_I2C_SCL_SOURCE,
76                  CAMERA_I2C_SCL_AF);
77  GPIO_PinAFConfig(CAMERA_I2C_SDA_GPIO_PORT, CAMERA_I2C_SDA_SOURCE,
```

```
78         CAMERA_I2C_SDA_AF);
79
80     /* 初始化 GPIO */
81     GPIO_InitStructure.GPIO_Pin = CAMERA_I2C_SCL_PIN ;
82     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
83     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_2MHz;
84     GPIO_InitStructure.GPIO_OType = GPIO_OType_OD;
85     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
86     GPIO_Init(CAMERA_I2C_SCL_GPIO_PORT, &GPIO_InitStructure);
87     GPIO_PinAFConfig(CAMERA_I2C_SCL_GPIO_PORT, CAMERA_I2C_SCL_SOURCE,
88                     CAMERA_I2C_SCL_AF);
89
90     GPIO_InitStructure.GPIO_Pin = CAMERA_I2C_SDA_PIN ;
91     GPIO_Init(CAMERA_I2C_SDA_GPIO_PORT, &GPIO_InitStructure);
92
93     /*初始化 I2C 模式 */
94     I2C_DeInit(CAMERA_I2C);
95
96     I2C_InitStruct.I2C_Mode = I2C_Mode_I2C;
97     I2C_InitStruct.I2C_DutyCycle = I2C_DutyCycle_2;
98     I2C_InitStruct.I2C_OwnAddress1 = 0xFE;
99     I2C_InitStruct.I2C_Ack = I2C_Ack_Enable;
100    I2C_InitStruct.I2C_AcknowledgedAddress = I2C_AcknowledgedAddress_7b
101                                           it;
102    I2C_InitStruct.I2C_ClockSpeed = 400000;
103
104    /* 写入配置 */
105    I2C_Init(CAMERA_I2C, &I2C_InitStruct);
106
107    /* 使能 I2C */
108    I2C_Cmd(CAMERA_I2C, ENABLE);
109
110    Delay(50); //延时 50ms
111 }
```

与所有使用到 GPIO 的外设一样，都要先把使用到的 GPIO 引脚模式初始化，以上代码把 DCMI 接口的信号线全都初始化为 DCMI 复用功能，**这里需要特别注意的地方是：**OV5640 的上电时序比较特殊，我们初始化 PWDN 和 RST 应该特别小心，先初始化成普通的推挽输出模式，并且在初始化完毕后直接控制 RST 为低电平，PWDN 为高电平，使摄像头处于待机模式，延时 10ms 后控制 PWDN 为低电平，再延时 10ms 后控制 RST 为高电平，OV5640 模组启动。

函数中还包含了 I2C 的初始化配置，使用 I2C 与 OV5640 的 SCCB 接口通讯，这里的 I2C 模式配置与标准的 I2C 无异。**特别注意：I2C 初始化完必须延时 50ms，再进行对 OV5640 寄存器的读写操作。**

配置 DCMI 的模式

接下来需要配置 DCMI 的工作模式，我们通过编写 OV5640_Init 函数完成该功能，见代码清单 54-4。

代码清单 54-4 配置 DCMI 的模式(bsp_ov5640.c 文件)

```
1
2 #define FSMC_LCD_ADDRESS      FSMC_Addr_ILI9806G_DATA
3
4
5 /**
6  * @brief 配置 DCMI/DMA 以捕获摄像头数据
7  * @param None
```

```
8  * @retval None
9  */
10 void OV5640_Init(void)
11 {
12     DCMI_InitTypeDef DCMI_InitStructure;
13     NVIC_InitTypeDef NVIC_InitStructure;
14
15     /*** 配置 DCMI 接口 ***/
16     /* 使能 DCMI 时钟 */
17     RCC_AHB2PeriphClockCmd(RCC_AHB2Periph_DCMI, ENABLE);
18
19     /* DCMI 配置*/
20     DCMI_InitStructure.DCMI_CaptureMode = DCMI_CaptureMode_Continuous;
21     DCMI_InitStructure.DCMI_SynchroMode = DCMI_SynchroMode_Hardware;
22     DCMI_InitStructure.DCMI_PCKPolarity = DCMI_PCKPolarity_Rising;
23     DCMI_InitStructure.DCMI_VSPolarity = DCMI_VSPolarity_Low;
24     DCMI_InitStructure.DCMI_HSPolarity = DCMI_HSPolarity_Low;
25     DCMI_InitStructure.DCMI_CaptureRate = DCMI_CaptureRate_All_Frame;
26     DCMI_InitStructure.DCMI_ExtendedDataMode = DCMI_ExtendedDataMode_8b;
27     DCMI_Init(&DCMI_InitStructure);
28
29     //配置 DMA 传输，直接配置循环传输即可
30     OV2640_DMA_Config(FSMC_LCD_ADDRESS,1);
31
32     /* 配置中断 */
33     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
34
35     /* 配置帧中断，接收到帧同步信号就进入中断 */
36     NVIC_InitStructure.NVIC_IRQChannel = DCMI_IRQn ; //帧中断
37     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
38     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
39     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
40     NVIC_Init(&NVIC_InitStructure);
41     DCMI_ITConfig (DCMI_IT_FRAME,ENABLE);
42 }
```

该函数的执行流程如下：

- (1) 使能 DCMI 外设的时钟，它是挂载在 AHB2 总线上的；
- (2) 根据摄像头的时序和硬件连接的要求，配置 DCMI 工作模式为：使用硬件同步，连续采集所有帧数据，采集时使用 8 根数据线，PIXCLK 被设置为上升沿有效，VSYNC 被设置成高电平有效，HSYNC 被设置成低电平有效；
- (3) 调用 OV5640_DMA_Config 函数开始 DMA 数据传输，每传输完一行数据需要调用一次，它包含本次传输的目的首地址及传输的数据量，后面我们再详细解释；
- (4) 配置 DCMI 的帧传输中断，可以用于统计帧率。另外，为了防止有时 DMA 出现传输错误或传输速度跟不上导致数据错位、偏移等问题，每次 DCMI 接收到摄像头的一帧数据，得到新的帧同步信号后(VSYNC)，就进入中断，重新设置液晶屏显示窗口，可使它重新开始一帧的数据传输（本工程没有实现这个功能）。

配置 DMA 数据传输

上面的 DCMI 配置函数中调用了 OV5640_DMA_Config 函数开始了 DMA 传输，该函数的定义见代码清单 54-5。

代码清单 54-5 配置 DMA 数据传输(bsp_ov5640.c 文件)

```
1 /**
2  * @brief 配置 DCMI/DMA 以捕获摄像头数据
3  * @param DMA_Memory0BaseAddr:本次传输的目的首地址
```



```
4  * @param DMA_BufferSize: 本次传输的数据量(单位为字,即 4 字节)
5  */
6  void OV5640_DMA_Config(uint32_t DMA_Memory0BaseAddr,uint16_t DMA_BufferSize)
7  {
8
9      DMA_InitTypeDef  DMA_InitStructure;
10
11     /* 配置 DMA 从 DCMI 中获取数据*/
12     /* 使能 DMA*/
13     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_DMA2, ENABLE);
14     DMA_Cmd(DMA2_Stream1,DISABLE);
15     while (DMA_GetCmdStatus(DMA2_Stream1) != DISABLE) {}
16
17     DMA_InitStructure.DMA_Channel = DMA_Channel_1;
18     DMA_InitStructure.DMA_PeripheralBaseAddr = DCMI_DR_ADDRESS; //DCMI 数据寄存器地址
19     //DMA 传输的目的地址(传入的参数)
20     DMA_InitStructure.DMA_Memory0BaseAddr = DMA_Memory0BaseAddr;
21     DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralToMemory;
22     DMA_InitStructure.DMA_BufferSize =DMA_BufferSize; //传输的数据大小(传入的参数)
23     DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
24     //液晶数据地址,不自增
25     DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Disable;
26     DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Word;
27     DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_HalfWord;
28     DMA_InitStructure.DMA_Mode = DMA_Mode_Circular; //循环模式
29     DMA_InitStructure.DMA_Priority = DMA_Priority_High;
30     DMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Enable;
31     DMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_Full;
32     DMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_INC8;
33     DMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_Single;
34
35     /*DMA 初始化 */
36     DMA_Init(DMA2_Stream1, &DMA_InitStructure);
37
38     DMA_Cmd(DMA2_Stream1,ENABLE);
39     while (DMA_GetCmdStatus(DMA2_Stream1) != ENABLE) {}
40 }
```

该函数跟普通的 DMA 配置无异,它把 DCMI 接收到的数据从它的数据寄存器搬运到液晶显示器上。它包含 2 个输入参数: DMA_Memory0BaseAddr 和 DMA_BufferSize,其中 DMA_Memory0BaseAddr 用于设置本次 DMA 传输的目的首地址,该参数会被赋值到结构体成员 DMA_InitStructure.DMA_Memory0BaseAddr 中。

根据前面的函数 OV5640_Init 对它的调用, DMA_Memory0BaseAddr 的值为 ILI9806G 液晶屏接收显存数据的地址,当 STM32 往该地址写入数据时,FSMC 会自动产生时序把该数据写入到液晶屏,从而可以使用液晶屏显示摄像头采集得的图像。

另一个参数 DMA_BufferSize 则用于指示本次 DMA 传输的数据量,它会被赋值到结构体成员 DMA_InitStructure.DMA_BufferSize 中,要注意它的单位是一个字,即 4 字节,是 DCMI 数据寄存器的单位大小。

根据前面的函数 OV5640_Init 对它的调用, DMA_BufferSize 的值被直接赋为 1,而 DMA 又被配置为循环模式,所以当开始传输时,STM32 会把摄像头采集到 DCMI 数据寄存器里的 4 字节数据使用 DMA 分 2 次写入到液晶屏数据地址,然后不断循环该过程。

若液晶屏提前调用 ILI9806G_OpenWindow 设置好了数据的显示窗口,那么这些数据就会一个一个(16 位的数据)地显示在对应的位置,从而实时显示摄像头采集得到的数据。

DCMI 帧中断

OV5640_Init 函数初始化了 DCMI，使能了帧中断，当摄像头采集到一帧图像的，会进入该中断，见代码清单 53-6 中的 DCMI_IRQHandler。

代码清单 54-6 DMA 传输完成中断与帧中断(stm32f4xx_it.c 文件)

```
1 extern uint8_t fps;
2
3
4 //使用帧中断重置 line_num,可防止有时掉数据的时候 DMA 传送行数出现偏移
5 void DCMI_IRQHandler(void)
6 {
7
8     if ( DCMI_GetITStatus (DCMI_IT_FRAME) == SET ) {
9         /*传输完一帧，计数复位*/
10         fps++; //帧率计数
11
12         DCMI_ClearITPendingBit(DCMI_IT_FRAME);
13     }
14 }
15
```

在本工程中的帧中断服务函数中，仅仅是使用了一个变量用于统计帧数，以便配合定时器计算帧率。实际上，为了避免有时 STM32 内部 DMA 传输受到阻塞而跟不上外部摄像头信号导致的数据错误（即液晶显示摄像头的的数据错位），可以在本中断服务函数中重新调用液晶屏的 ILI9806G_OpenWindow 复位液晶显示的起始位置，达到重新同步的目的，由于本工程实测并没有错位的问题，所以这里省略了，当您了解了整个工程的原理后，可以自行添加防错位的处理代码。

使能 DCMI 采集

以上是我们使用 DCMI 的传输配置，但它还没有使能 DCMI 采集，在实际使用中还需要调用下面两个库函数开始采集数据。

```
1 //使能 DCMI 采集数据
2 DCMI_Cmd(ENABLE);
3 DCMI_CaptureCmd(ENABLE);
```

读取 OV5640 芯片 ID

配置完了 STM32 的 DCMI，还需要控制摄像头，它有很多寄存器用于配置工作模式。利用 STM32 的 I2C 接口，可向 OV5640 的寄存器写入控制参数，我们先写个读取芯片 ID 的函数测试一下，见代码清单 54-7。

代码清单 54-7 读取 OV5640 的芯片 ID(bsp_ov5640.c 文件)

```
1 //存储摄像头 ID 的结构体
2 typedef struct {
3     uint8_t PIDH;
4     uint8_t PIDL;
5 } OV5640_IDTypeDef;
6 #define OV5640_SENSOR_PIDH      0x300A
7 #define OV5640_SENSOR_PIDL      0x300B
8 /**
9  * @brief  读取摄像头的 ID.
10  * @param  OV5640ID: 存储 ID 的结构体
11  * @retval None
12  */
13 void OV5640_ReadID(OV5640_IDTypeDef *OV5640ID)
```

```
14 {
15
16     /*读取寄存芯片 ID*/
17     OV5640ID->PIDH = OV5640_ReadReg(OV5640_SENSOR_PIDH);
18     OV5640ID->PIDL = OV5640_ReadReg(OV5640_SENSOR_PIDL);
19 }
20 /**
21  * @brief 从OV5640 寄存器中读取一个字节的数据
22  * @param Addr: 寄存器地址
23  * @retval 返回读取的数据
24  */
25 uint8_t OV5640_ReadReg(uint16_t Addr)
26 {
27     uint32_t timeout = DCMI_TIMEOUT_MAX;
28     uint8_t Data = 0;
29
30     /* Generate the Start Condition */
31     I2C_GenerateSTART(CAMERA_I2C, ENABLE);
32
33     /* Test on CAMERA_I2C EV5 and clear it */
34     timeout = DCMI_TIMEOUT_MAX; /* Initialize timeout value */
35     while (!I2C_CheckEvent(CAMERA_I2C, I2C_EVENT_MASTER_MODE_SELECT)) {
36         /* If the timeout delay is exeeded, exit with error code */
37         if ((timeout--) == 0) return 0xFF;
38     }
39
40     /* Send DCMI selcted device slave Address for write */
41     I2C_Send7bitAddress(CAMERA_I2C, OV5640_DEVICE_ADDRESS,
42                         I2C_Direction_Transmitter);
43
44     /* Test on I2C1 EV6 and clear it */
45     timeout = DCMI_TIMEOUT_MAX; /* Initialize timeout value */
46     while (!I2C_CheckEvent(CAMERA_I2C,
47                             I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED)) {
48         /* If the timeout delay is exeeded, exit with error code */
49         if ((timeout--) == 0) return 0xFF;
50     }
51
52     /* Send I2C1 location address MSB */
53     I2C_SendData(CAMERA_I2C, (uint8_t)((Addr>>8) & 0xFF) );
54
55     /* Test on I2C1 EV8 and clear it */
56     timeout = DCMI_TIMEOUT_MAX; /* Initialize timeout value */
57     while (!I2C_CheckEvent(CAMERA_I2C,
58                             I2C_EVENT_MASTER_BYTE_TRANSMITTED)) {
59         /* If the timeout delay is exeeded, exit with error code */
60         if ((timeout--) == 0) return 0xFF;
61     }
62
63     /* Clear AF flag if arised */
64     CAMERA_I2C->SR1 |= (uint16_t)0x0400;
65
66     //-----
67     /* Send I2C1 location address LSB */
68     I2C_SendData(CAMERA_I2C, (uint8_t)(Addr & 0xFF) );
69     /* Test on I2C1 EV8 and clear it */
70     timeout = DCMI_TIMEOUT_MAX; /* Initialize timeout value */
71     while (!I2C_CheckEvent(CAMERA_I2C,
72                             I2C_EVENT_MASTER_BYTE_TRANSMITTED)) {
73         /* If the timeout delay is exeeded, exit with error code */
74         if ((timeout--) == 0) return 0xFF;
75     }
76
77     /* Clear AF flag if arised */
78     CAMERA_I2C->SR1 |= (uint16_t)0x0400;
79     //-----
80 }
```

```
81      /* Generate the Start Condition */
82      I2C_GenerateSTART(CAMERA_I2C, ENABLE);
83
84      /* Test on I2C1 EV6 and clear it */
85      timeout = DCMI_TIMEOUT_MAX; /* Initialize timeout value */
86      while (!I2C_CheckEvent(CAMERA_I2C, I2C_EVENT_MASTER_MODE_SELECT)) {
87          /* If the timeout delay is exceeded, exit with error code */
88          if ((timeout--) == 0) return 0xFF;
89      }
90
91      /* Send DCMI selcted device slave Address for write */
92      I2C_Send7bitAddress(CAMERA_I2C, OV5640_DEVICE_ADDRESS,
93                          I2C_Direction_Receiver);
94
95      /* Test on I2C1 EV6 and clear it */
96      timeout = DCMI_TIMEOUT_MAX; /* Initialize timeout value */
97      while (!I2C_CheckEvent(CAMERA_I2C,
98                             I2C_EVENT_MASTER_RECEIVER_MODE_SELECTED)) {
99          /* If the timeout delay is exceeded, exit with error code */
100         if ((timeout--) == 0) return 0xFF;
101     }
102
103     /* Prepare an NACK for the next data received */
104     I2C_AcknowledgeConfig(CAMERA_I2C, DISABLE);
105
106     /* Test on I2C1 EV7 and clear it */
107     timeout = DCMI_TIMEOUT_MAX; /* Initialize timeout value */
108     while (!I2C_CheckEvent(CAMERA_I2C, I2C_EVENT_MASTER_BYTE_RECEIVED))
109     {
110         /* If the timeout delay is exceeded, exit with error code */
111         if ((timeout--) == 0) return 0xFF;
112     }
113
114     /* Prepare Stop after receiving data */
115     I2C_GenerateSTOP(CAMERA_I2C, ENABLE);
116
117     /* Receive the Data */
118     Data = I2C_ReceiveData(CAMERA_I2C);
119
120     /* return the read data */
121     return Data;
122 }
```

在 OV5640 的 PIDH 及 PIDL 寄存器存储了产品 ID，PIDH 的默认值为 0x56，PIDL 的默认值为 0x40。在代码中我们定义了一个结构体 OV5640_IDTypeDef 专门存储这些读取得的 ID 信息。

OV5640_ReadID 函数中使用的 OV5640_ReadReg 函数是使用 STM32 的 I2C 外设向某寄存器读写单个字节数据的底层函数，它与我们前面章节中用到的 I2C 函数差异是 OV5640 的寄存器地址是 16 位的。程序中是先发高 8 位地址接着发低 8 位地址，再读取寄存器的值。

向 OV5640 写入寄存器配置

检测到 OV5640 的存在后，向它写入配置参数，见代码清单 54-8。

代码清单 54-8 向 OV5640 写入寄存器配置

```
1
2 unsigned short RGB565_Init[][2]= {
3     //15fps VGA RGB565 output
4     // 24MHz input clock, 24MHz PCLK
5     0x3103, 0x11, // system clock from pad, bit[1]
6     0x3008, 0x82, // software reset, bit[7]
```

```
7 // delay 5ms
8 0x3008, 0x42, // software power down, bit[6]
9 0x3103, 0x03, // system clock from PLL, bit[1]
10 0x3017, 0xff, // FREX, Vsync, HREF, PCLK, D[9:6] output enable
11 0x3018, 0xff, // D[5:0], GPIO[1:0] output enable
12 0x3034, 0x1a, // MIPI 10-bit
13 0x3037, 0x13, // PLL root divider, bit[4], PLL pre-divider, bit[3:0]
14 /*以下配置省略*/
15 }
16
17 unsigned short sensor_reg[(sizeof(RGB565_Init)/4)];
18 /**
19  * @brief 配置 OV5640 为 BMP 模式.
20  * @retval None
21  */
22 void OV5640_RGB565_Default_Config(void)
23 {
24     uint32_t i;
25
26     /*摄像头复位*/
27     OV5640_Reset();
28
29     Delay(50);
30
31     /* 写入寄存器配置 RGB565 模式 */
32     for (i=0; i<(sizeof(RGB565_Init)/4); i++) {
33         OV5640_WriteReg(RGB565_Init[i][0], RGB565_Init[i][1]);
34
35         Delay(5);
36         sensor_reg[i] = OV5640_ReadReg(RGB565_Init[i][0]);
37
38         CAMERA_DEBUG("sensor_reg[0x%x]:%x-%x\n", RGB565_Init[i][0], RGB565_Init[i][1], sensor_reg[i]);
39     }
40
41     Delay(100);
42
43     //以默认模式初始化
44     for (i=0; i<(sizeof(RGB565_WVGA)/2); i++) {
45         OV5640_WriteReg(RGB565_WVGA[i][0], RGB565_WVGA[i][1]);
46     }
47 }
48
49
50
51
52 /**
53  * @brief 写一字节数据到 OV5640 寄存器
54  * @param Addr: OV5640 的寄存器地址
55  * @param Data: 要写入的数据
56  * @retval 返回 0 表示写入正常, 0xFF 表示错误
57  */
58 uint8_t OV5640_WriteReg(uint16_t Addr, uint8_t Data)
59 {
60     uint32_t timeout = DCMI_TIMEOUT_MAX;
61
62     /* Generate the Start Condition */
63     I2C_GenerateSTART(CAMERA_I2C, ENABLE);
64
65     /* Test on CAMERA_I2C EV5 and clear it */
66     timeout = DCMI_TIMEOUT_MAX; /* Initialize timeout value */
67     while (!I2C_CheckEvent(CAMERA_I2C, I2C_EVENT_MASTER_MODE_SELECT)) {
68         /* If the timeout delay is exceeded, exit with error code */
69         if ((timeout--) == 0) return 0xFF;
70     }
71
72     /* Send DCMI selected device slave Address for write */
73     I2C_Send7bitAddress(CAMERA_I2C, OV5640_DEVICE_ADDRESS, I2C_Direction_Transmitter);
```

```
74
75     /* Test on CAMERA_I2C EV6 and clear it */
76     timeout = DCMI_TIMEOUT_MAX; /* Initialize timeout value */
77     while (!I2C_CheckEvent(CAMERA_I2C, I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED)) {
78         /* If the timeout delay is exeeded, exit with error code */
79         if ((timeout--) == 0) return 0xFF;
80     }
81
82     /* Send CAMERA_I2C location address MSB */
83     I2C_SendData(CAMERA_I2C, (uint8_t)( (Addr >> 8) & 0xFF) );
84
85     /* Test on CAMERA_I2C EV8 and clear it */
86     timeout = DCMI_TIMEOUT_MAX; /* Initialize timeout value */
87     while (!I2C_CheckEvent(CAMERA_I2C, I2C_EVENT_MASTER_BYTE_TRANSMITTED)) {
88         /* If the timeout delay is exeeded, exit with error code */
89         if ((timeout--) == 0) return 0xFF;
90     }
91     //-----
92     /* Send I2C1 location address LSB */
93     I2C_SendData( CAMERA_I2C, (uint8_t)(Addr & 0xFF) );
94     /* Test on I2C1 EV8 and clear it */
95     timeout = DCMI_TIMEOUT_MAX; /* Initialize timeout value */
96     while (!I2C_CheckEvent(CAMERA_I2C, I2C_EVENT_MASTER_BYTE_TRANSMITTED)) {
97         /* If the timeout delay is exeeded, exit with error code */
98         if ((timeout--) == 0) return 0xFF;
99     }
100
101
102     //-----
103
104     /* Send Data */
105     I2C_SendData(CAMERA_I2C, Data);
106
107     /* Test on CAMERA_I2C EV8 and clear it */
108     timeout = DCMI_TIMEOUT_MAX; /* Initialize timeout value */
109     while (!I2C_CheckEvent(CAMERA_I2C, I2C_EVENT_MASTER_BYTE_TRANSMITTED)) {
110         /* If the timeout delay is exeeded, exit with error code */
111         if ((timeout--) == 0) return 0xFF;
112     }
113
114     /* Send I2C1 STOP Condition */
115     I2C_GenerateSTOP(CAMERA_I2C, ENABLE);
116
117     /* If operation is OK, return 0 */
118     return 0;
119 }
```

这个 OV5640_RGB565Config 函数直接把一个初始化的二维数组 RGB565_Init 和一个分辨率设置的二维数组 RGB565_WVGA 使用 I2C 传输到 OV5640 中，二维数组的第一维存储的是寄存器地址，第二维存储的是对应寄存器要写入的控制参数。OV5640_WriteReg 函数中，因为 OV5640 的寄存器地址为 16 位，所以写寄存器的时候会先写入高 8 位的地址接着写入低 8 位的地址，然后再写入寄存器的值，这个是有别于普通的 I2C 设备的写入方式，需要特别注意。

如果您对这些寄存器配置感兴趣，可以一个个对着 OV5640 的寄存器说明来阅读，这些配置主要是把 OV5640 配置成了 WVGA 时序模式，并使用 8 根数据线输出格式为 RGB565 的图像数据。我们参考《OV5640_自动对焦照相模组应用指南(DVP_接口)_R2.13C.pdf》文档中第 20 页 4.1.3 节的 800x480（本配置为 854*480，与说明的类似）

预览的寄存器参数进行配置。使摄像头输出为 WVGA 模式。

4.1.3 800x480 预览

800x480 15 帧/秒

```
// 800x480 15fps, night mode 5fps
// input clock 24Mhz, PCLK 45.6Mhz
write_i2c(0x3035, 0x41);    // PLL
write_i2c(0x3036, 0x72);    // PLL
write_i2c(0x3c07, 0x08);    // light meter 1 threshold[7:0]
write_i2c(0x3820, 0x41);    // flip
write_i2c(0x3821, 0x07);    // mirror
write_i2c(0x3814, 0x31);    // timing X inc
```

初始化 OV5640 自动对焦功能

写入 OV5640 的配置参数后，需要向它写入自动对焦固件，初始化自动对焦功能，才能使用自动对焦功能，见代码清单 54-9。

代码清单 54-9 初始化 OV5640 自动对焦功能

```
1 void OV5640_AUTO_FOCUS(void)
2 {
3     OV5640_FOCUS_AD5820_Init();
4     OV5640_FOCUS_AD5820_Constant_Focus();
5 }
6 static void OV5640_FOCUS_AD5820_Init(void)
7 {
8     u8 state=0x8F;
9     u32 iteration = 100;
10    u16 totalCnt = 0;
11
12    CAMERA_DEBUG("OV5640_FOCUS_AD5820_Init\n");
13
14    OV5640_WriteReg(0x3000, 0x20);
15    totalCnt = sizeof(OV5640_AF_FW);
16    CAMERA_DEBUG("Total Count = %d\n", totalCnt);
17
18    // 写入自动对焦固件 Brust mode
19    OV5640_WriteFW(OV5640_AF_FW, totalCnt);
20
21    OV5640_WriteReg(0x3022, 0x00);
22    OV5640_WriteReg(0x3023, 0x00);
23    OV5640_WriteReg(0x3024, 0x00);
24    OV5640_WriteReg(0x3025, 0x00);
25    OV5640_WriteReg(0x3026, 0x00);
26    OV5640_WriteReg(0x3027, 0x00);
27    OV5640_WriteReg(0x3028, 0x00);
28    OV5640_WriteReg(0x3029, 0xFF);
29    OV5640_WriteReg(0x3000, 0x00);
30    OV5640_WriteReg(0x3004, 0xFF);
31    OV5640_WriteReg(0x0000, 0x00);
32    OV5640_WriteReg(0x0000, 0x00);
33    OV5640_WriteReg(0x0000, 0x00);
34    OV5640_WriteReg(0x0000, 0x00);
35
36    do {
37        state = (u8)OV5640_ReadReg(0x3029);
38        CAMERA_DEBUG("when init af, state=0x%x\n", state);
39
40        Delay(10);
```

```
41     if (iteration-- == 0) {
42         CAMERA_DEBUG("[OV5640]STA_FOCUS state check ERROR!!,\n", state);
43         break;
44     }
45 } while (state!=0x70);
46
47 OV5640_FOCUS_AD5820_Check_MCU();
48 return;
49 } /* OV5640_FOCUS_AD5820_Init */
50
51 //set constant focus
52 void OV5640_FOCUS_AD5820_Constant_Focus(void)
53 {
54     u8 state = 0x8F;
55     u32 iteration = 300;
56     //send constant focus mode command to firmware
57     OV5640_WriteReg(0x3023,0x01);
58     OV5640_WriteReg(0x3022,0x04);
59
60     iteration = 5000;
61     do {
62         state = (u8)OV5640_ReadReg(0x3023);
63         if (iteration-- == 0) {
64             CAMERA_DEBUG("[OV5640]AD5820_Single_Focus time out !!\n", state);
65             return ;
66         }
67         Delay(10);
68     } while (state!=0x00); //0x0 : focused 0x01: is focusing
69     return;
70 }
71
72 }
```

OV5640_AUTO_FOCUS 函数调用了 OV5640_FOCUS_AD5820_Init 函数和 OV5640_FOCUS_AD5820_Constant_Focus 函数，我们先来介绍 OV5640_FOCUS_AD5820_Init 函数，首先复位 OV5640 内部的 MCU，然后通过 I2C 的突发模式写入自动对焦固件，突发模式就是只需要写入首地址，接着就一直写数据，这个过程地址会自增，直接写完数据位置，对于连续地址写入相当方便。写入固件之后 OV5640 内部 MCU 开始初始化，最后检查初始化完成的状态是否为 0x70，如果是就代表固件已经写入成功，并初始化成功。接着，我们需要 OV5640_FOCUS_AD5820_Constant_Focus 函数来调用自动对焦固件中的持续对焦指令，完成以上步骤后，摄像头就已经初始化完毕。

OV5640 的其它模式配置

以上是最基本的摄像头采集过程，而提供的工程在以上基础还增加了一些摄像头的配置，包括分辨率、光照度、饱和度、对比度及特殊模式等，如 OV5640_OutSize_Set、OV5640_BrightnessConfig、OV5640_Color_Saturation、OV5640_ContrastConfig 和 OV5640_SpecialEffects 等函数，这些函数的本质都是根据函数的输入参数，转化成对应的配置写入到 OV5640 摄像头的寄存器中，完成相应的配置，下面仅以 OV5640_SpecialEffects 函数为例进行介绍，见代码清单 54-10。

代码清单 54-10 OV5640_SpecialEffects 函数

```
1
2 //特效设置参数表
3 const static uint8_t OV5640_Effect_reg[][4]= {
4     0x06,0x40,0x10,0x08, //正常
5     0x1E,0xA0,0x40,0x08, //冷色
6     0x1E,0x80,0xC0,0x08, //暖色
```

```

7      0X1E,0x80,0X80,0X08, // 黑白
8      0X1E,0x40,0XA0,0X08, // 泛黄
9      0X40,0x40,0X10,0X08, // 反色
10     0X1E,0x60,0X60,0X08, // 偏绿
11     0X1E,0xF0,0XF0,0X08, // 过曝
12     0X06,0x40,0X10,0X09, // 正负片叠加
13 };
14 /**
15  * @brief 特殊效果
16  * @param 参数用于选择光线模式
17  *      0x01 正常
18  *      0x02 冷色
19  *      0x03 暖色
20  *      0x04 黑白
21  *      0x05 泛黄
22  *      0x06 反色
23  *      0x07 偏绿
24  *      0x08 过曝
25  *      0x09 正负片叠加
26
27  * @retval None
28  */
29 void OV5640_SpecialEffects(uint8_t mode)
30 {
31     OV5640_WriteReg(0x3212, 0x03); // start group 3
32
33     OV5640_WriteReg(0x5580, OV5640_Effect_reg[mode][0]);
34     OV5640_WriteReg(0x5583, OV5640_Effect_reg[mode][1]); // sat U
35     OV5640_WriteReg(0x5584, OV5640_Effect_reg[mode][2]); // sat V
36     OV5640_WriteReg(0x5003, OV5640_Effect_reg[mode][3]);
37
38     OV5640_WriteReg(0x3212, 0x13); // end group 3
39     OV5640_WriteReg(0x3212, 0xa3); // launch group 3
40 }

```

从代码中可了解到，函数支持 1~9 作为输入参数，分别对应不同的模式，在函数内部根据不同的输入对寄存器写入相应配置。

设置液晶屏显示窗口

初始化摄像头后，即使可以启用 DCMI 采集，并且设置液晶显示窗口，然后 DMA 会把数据传输至液晶屏上，见代码清单 53-9。

代码清单 54-11 设置液晶显示窗口(main.c)

```

1  /**
2   * @brief 液晶屏开窗，使能摄像头数据采集
3   * @param 无
4   * @retval 无
5   */
6  void ImagDisp(void)
7  {
8      //扫描模式，横屏
9      ILI9806G_GramScan(cam_mode.lcd_scan);
10     LCD_SetFont(&Font16x32);
11     LCD_SetColors(RED,BLACK);
12
13     ILI9806G_Clear(0,0,LCD_X_LENGTH,LCD_Y_LENGTH); /* 清屏，显示全黑 */
14
15     /*DMA 会把数据传输到液晶屏，开窗后数据按窗口排列 */
16     ILI9806G_OpenWindow(cam_mode.lcd_sx,
17                          cam_mode.lcd_sy,
18                          cam_mode.cam_out_width,

```

```
19             cam_mode.cam_out_height);  
20  
21     OV5640_Capture_Control(ENABLE);  
22 }
```

在本函数中，先调用了 ILI9806G_GramScan 函数把液晶屏设置成横屏显示模式，然后使用 ILI9806G_OpenWindow 设置一个起始位置为 lcd_sx, lcd_sy 宽 cam_out_width 高 cam_out_height 的窗口(它们是结构体 cam_mode 的成员，稍后介绍)，根据液晶驱动 FSMC 的配置，开窗后只要我们往地址 FSMC_LCD_ADDRESS 写入的数据会一个一个地添加到该窗口中进行显示，而前面的代码已经配置了 DMA 搬运摄像头数据的工作，只要执行完本函数最后的 OV5640_Capture_Control 使能 DCMI 采集，一切就会开始正常运转：摄像头根据寄存器的配置输出 RGB565 的图像，STM32 的 DCMI 外设把接收到的图像数据缓存到 DCMI 数据寄存器中，当接收到 2 个像素数据后，DMA 开始传输，把这两个数据搬运到内存地址 FSMC_LCD_ADDRESS，往该地址写入数据时触发 FSMC 产生时序写入到液晶屏，最终液晶屏把数据显示到窗口上，如此循环往复，完成一帧数据的采集和显示，不断循环，即可实现连续的实时采集与显示。

摄像头配置结构体

由于分辨率、光照度、饱和度、对比度及特殊模式等摄像头配置涉及众多内容，特别是关于分辨率的配置，需要与液晶扫描方向匹配，否则容易出现显示错误的现象，为了方便使用，工程中定义了一个结构体类型专门用于设置摄像头的这些配置，在初始化摄像头或想修改配置的时候，修改该变量的内容，然后把它作为参数输入到各种配置函数调用即可。该摄像头配置结构体类型见代码清单 54-12。

代码清单 54-12 摄像头配置结构体 (bsp_ov5640.h 文件)

```
1  
2  /*摄像头配置结构体*/  
3  typedef struct {  
4      uint8_t frame_rate; //输出帧率  
5  
6      uint16_t cam_isp_sx; //摄像头 ISP X 起始位置  
7      uint16_t cam_isp_sy; //摄像头 ISP Y 起始位置  
8  
9      uint16_t cam_isp_width; //摄像头 ISP 宽  
10     uint16_t cam_isp_height; //摄像头 ISP 高  
11  
12     uint8_t scaling; //是否使用自动缩放，推荐使用，1:使用，0:不使用  
13  
14     uint16_t cam_out_sx; //摄像头输出窗口 X 起始位置  
15     uint16_t cam_out_sy; //摄像头输出窗口 Y 起始位置  
16  
17     uint16_t cam_out_width; //输出图像分辨率，宽  
18     uint16_t cam_out_height; //输出图像分辨率，高  
19  
20     uint16_t lcd_sx; //图像显示在液晶屏的 X 起始位置  
21     uint16_t lcd_sy; //图像显示在液晶屏的 Y 起始位置  
22     uint8_t lcd_scan; //液晶屏的扫描模式 (0~7)  
23  
24     uint8_t light_mode; //光照模式，参数范围 [0~4]  
25     int8_t saturation; //饱和度，参数范围 [-3 ~ +3]  
26     int8_t brightness; //光照度，参数范围 [-4~+4]  
27     int8_t contrast; //对比度，参数范围 [-3~+3]  
28     uint8_t effect; //特殊效果，参数范围 [0~9]:
```

```
29     int8_t exposure;//曝光补偿, 参数范围[-3~+3]
30
31     uint8_t auto_focus;//是否使用自动对焦功能 1:使用, 0:不使用
32
33 } OV5640_MODE_PARAM;
```

结构体类型定义中的代码注释有注明各种配置参数的范围, 如其中的 frame_rate 可以配置摄像头的输出帧率, 可以赋值为 FRAME_RATE_15FPS 或 FRAME_RATE_30FPS 的宏, 分辨率高时可能会出现图像花屏的现象。

在工程中, 提供了三组摄像头及显示配置范例, 实验时可以亲自尝试一下来了解, 见代码清单 54-13。

代码清单 54-13 三组摄像头初始化配置 (bsp_ov5640.c 文件)

```
1 //摄像头初始化配置
2 //注意: 使用这种方式初始化结构体, 要在 c/c++选项中选择 C99 mode
3 OV5640_MODE_PARAM cam_mode = {
4
5     /*以下包含几组摄像头配置, 可自行测试, 保留一组, 把其余配置注释掉即可*/
6     /*****配置 1****854*480****横屏显示*****/
7
8     .frame_rate = FRAME_RATE_15FPS,
9
10    //ISP 窗口
11    .cam_isp_sx = 0,
12    .cam_isp_sy = 0,
13
14    .cam_isp_width = 1920,
15    .cam_isp_height = 1080,
16
17    //输出窗口
18    .scaling = 1, //使能自动缩放
19    .cam_out_sx = 16, //使能自动缩放后, 一般配置成 16 即可
20    .cam_out_sy = 4, //使能自动缩放后, 一般配置成 4 即可
21    .cam_out_width = 854,
22    .cam_out_height = 480,
23
24    //LCD 位置
25    .lcd_sx = 0,
26    .lcd_sy = 0,
27    .lcd_scan = 5, //LCD 扫描模式, 本横屏配置可用 1、3、5、7 模式
28
29    //以下可根据自己的需要调整, 参数范围见结构体类型定义
30    .light_mode = 0, //自动光照模式
31    .saturation = 0,
32    .brightness = 0,
33    .contrast = 0,
34    .effect = 0, //正常模式
35    .exposure = 0,
36
37    .auto_focus = 1,
38
39    /*****配置 2****240*320****竖屏显示*****/
40    // .frame_rate = FRAME_RATE_30FPS,
41    //
42    // //ISP 窗口
43    // .cam_isp_sx = 0,
44    // .cam_isp_sy = 0,
45    //
46    // .cam_isp_width = 1920,
47    // .cam_isp_height = 1080,
48    //
```

```

49 // //输出窗口
50 // .scaling = 1, //使能自动缩放
51 // .cam_out_sx = 16, //使能自动缩放后，一般配置成 16 即可
52 // .cam_out_sy = 4, //使能自动缩放后，一般配置成 4 即可
53 // .cam_out_width = 240,
54 // .cam_out_height = 320,
55 //
56 // //LCD 位置
57 // .lcd_sx = 120,
58 // .lcd_sy = 267,
59 // .lcd_scan = 6, //LCD 扫描模式,
60 //
61 // //以下可根据自己的需要调整，参数范围见结构体类型定义
62 // .light_mode = 0, //自动光照模式
63 // .saturation = 0,
64 // .brightness = 0,
65 // .contrast = 0,
66 // .effect = 0, //正常模式
67 // .exposure = 0,
68
69 // .auto_focus = 1, //自动对焦
70
71 //*****配置 3*****640*480****小分辨率*****
72 // .frame_rate = FRAME_RATE_30FPS,
73 //
74 // //ISP 窗口
75 // .cam_isp_sx = 0,
76 // .cam_isp_sy = 0,
77 //
78 // .cam_isp_width = 1920,
79 // .cam_isp_height = 1080,
80 //
81 // //输出窗口
82 // .scaling = 0, //使能自动缩放
83 // .cam_out_sx = 16, //使能自动缩放后，一般配置成 16 即可
84 // .cam_out_sy = 4, //使能自动缩放后，一般配置成 4 即可
85 // .cam_out_width = 640,
86 // .cam_out_height = 480,
87 //
88 // //LCD 位置
89 // .lcd_sx = 100,
90 // .lcd_sy = 0,
91 // .lcd_scan = 5, //LCD 扫描模式,
92 //
93 // //以下可根据自己的需要调整，参数范围见结构体类型定义
94 // .light_mode = 0, //自动光照模式
95 // .saturation = 0,
96 // .brightness = 0,
97 // .contrast = 0,
98 // .effect = 0, //正常模式
99 // .exposure = 0,
100
101 // .auto_focus = 1, //自动对焦
102
103 };

```

代码中使用 `OV5640_MODE_PARAM` 类型定义了一个 `cam_mode` 全局变量并对其结构体成员赋予了初始值。本工程默认使用以上第一组配置，采集 854*480 的图像在液晶屏上横屏显示；而第二组配置是 240*320 的 VGA 图像在液晶屏上竖屏显示，注意两组配置中 `cam_out_width`、`cam_out_height` 和 `lcd_mode` 变量值的区别；第三组配置是 640*480 的分辨

率，帧率为 30 帧。你可亲自尝试以上各组配置，使用时注释掉其余两组即可，也可以在范例的基础上，自己进行修改测试。

使用摄像头配置结构体时，在初始化摄像头时要调用相应的函数对寄存器进行赋值，本工程中定义了一个函数专门用于把 cam_mode 的所有配置更新到摄像头，见代码清单 54-14。

代码清单 54-14 OV5640_USER_Config 函数

```
1
2 /**
3  * @brief 使用 cammode 参数初始化各项配置
4  * @param None
5  * @retval None
6  */
7 void OV5640_USER_Config(void)
8 {
9     OV5640_FrameRate_Set(cam_mode.frame_rate);
10    Delay(100);
11
12    OV5640_ISPSize_Set(cam_mode.cam_isp_sx,
13                       cam_mode.cam_isp_sy,
14                       cam_mode.cam_isp_width,
15                       cam_mode.cam_isp_height);
16    Delay(100);
17
18    OV5640_OutSize_Set(cam_mode.scaling,
19                      cam_mode.cam_out_sx,
20                      cam_mode.cam_out_sy,
21                      cam_mode.cam_out_width,
22                      cam_mode.cam_out_height);
23    Delay(100);
24    OV5640_BrightnessConfig(cam_mode.brightness);
25    Delay(100);
26
27    OV5640_Color_Saturation(cam_mode.saturation);
28    Delay(100);
29
30    OV5640_ContrastConfig(cam_mode.contrast);
31    Delay(100);
32
33    OV5640_Exposure(cam_mode.exposure);
34    Delay(100);
35
36    OV5640_LightMode(cam_mode.light_mode);
37    Delay(100);
38
39    OV5640_SpecialEffects(cam_mode.effect);
40    Delay(100);
41
42    // Delay(500);
43 }
```

本函数根据全局变量 cam_mode 的值，依次调用对应的摄像头配置函数，把 cam_mode 的配置更新到摄像头寄存器中，所以，当程序运行时需要修改摄像头配置时，修改全局变量 cam_mode 的值后，调用 OV5640_USER_Config 函数即可。

main 函数

最后我们来编写 main 函数，利用前面讲解的函数，控制采集图像，见代码清单 54-15。

代码清单 54-15 main 函数

```
1
```

```
2 //修改 cam_mode 参数, 修改后需要重新调用配置函数才生效
3 void Camera_Mode_Reconfig(void)
4 {
5     cam_mode.frame_rate = FRAME_RATE_30FPS,
6
7     //ISP 窗口
8     cam_mode.cam_isp_sx = 0;
9     cam_mode.cam_isp_sy = 0;
10
11     cam_mode.cam_isp_width = 1920;
12     cam_mode.cam_isp_height = 1080;
13
14     //输出窗口
15     cam_mode.scaling = 1; //使能自动缩放
16     cam_mode.cam_out_sx = 16; //使能自动缩放后, 一般配置成 16 即可
17     cam_mode.cam_out_sy = 4; //使能自动缩放后, 一般配置成 4 即可
18     cam_mode.cam_out_width = 320;
19     cam_mode.cam_out_height = 240;
20
21     //LCD 位置
22     cam_mode.lcd_sx = 270;
23     cam_mode.lcd_sy = 120;
24     cam_mode.lcd_scan = 5; //LCD 扫描模式,
25
26     //以下可根据自己的需要调整, 参数范围见结构体类型定义
27     cam_mode.light_mode = 0; //自动光照模式
28     cam_mode.saturation = 0;
29     cam_mode.brightness = 0;
30     cam_mode.contrast = 0;
31     cam_mode.effect = 0; //正常模式
32     cam_mode.exposure = 0;
33
34     cam_mode.auto_focus = 1; //自动对焦
35 }
36
37
38 //【*】注意事项:
39 //本程序暂时不支持摄像头延长线!!!
40 /**
41  * @brief 主函数
42  * @param 无
43  * @retval 无
44  */
45 int main(void)
46 {
47     uint8_t focus_status = 0;
48
49     ILI9806G_Init (); //LCD 初始化
50
51     LCD_SetFont (&Font16x32);
52     LCD_SetColors (RED, BLACK);
53
54     ILI9806G_Clear (0,0,LCD_X_LENGTH,LCD_Y_LENGTH); /* 清屏, 显示全黑 */
55
56     Debug_USART_Config();
57
58     /* 配置 SysTick 为 1ms 中断一次, 时间到后触发定时中断,
59     * 进入 stm32fxx_it.c 文件的 SysTick_Handler 处理, 通过数中断次数计时
60     */
61     SysTick_Init();
62
63     Key_GPIO_Config();
64
65     //液晶扫描方向
66     ILI9806G_GramScan (5);
```

```
67
68 CAMERA_DEBUG("STM32F407 DCMI 驱动 OV5640 例程");
69
70 /* 初始化摄像头 GPIO 及 IIC */
71 OV5640_HW_Init();
72
73 /* 读取摄像头芯片 ID, 确定摄像头正常连接 */
74 OV5640_ReadID(&OV5640_Camera_ID);
75
76 if (OV5640_Camera_ID.PIDH == 0x56) {
77     sprintf((char*)dispBuf, "OV5640 Camera ID:0x%x,initializing... ", OV5640_Camera_ID.PIDH);
78     ILI9806G_DispStringLine_EN(LINE(0), dispBuf);
79     CAMERA_DEBUG("检测到摄像头 %x %x", OV5640_Camera_ID.PIDH, OV5640_Camera_ID.PIDL);
80
81 } else {
82     LCD_SetTextColor(RED);
83     ILI9806G_DispString_EN(10,10, "Can not detect OV5640 module,please check the connection!");
84     CAMERA_DEBUG("没有检测到 OV5640 摄像头, 请重新检查连接。");
85
86     while (1);
87 }
88
89
90 OV5640_Init();
91
92 OV5640_RGB565_Default_Config();
93
94 OV5640_USER_Config();
95
96 OV5640_FOCUS_AD5820_Init();
97
98
99 if (cam_mode.auto_focus ==1) {
100     OV5640_FOCUS_AD5820_Constant_Focus();
101     focus_status = 1;
102 }
103
104 /*DMA 直接传输摄像头数据到 LCD 屏幕显示*/
105 ImagDisp();
106
107 while (1) {
108     //运行时动态修改摄像头参数示例
109     if ( Key_Scan(KEY1_GPIO_PORT,KEY1_PIN) == KEY_ON ) {
110         //关闭采集
111         OV5640_Capture_Control(DISABLE);
112
113         //修改 Cam mode 参数
114         Camera_Mode_Reconfig();
115
116         OV5640_USER_Config();
117
118         if (cam_mode.auto_focus ==1) {
119             OV5640_FOCUS_AD5820_Constant_Focus();
120             focus_status = 1;
121         }
122
123         /*DMA 直接传输摄像头数据到 LCD 屏幕显示*/
124         ImagDisp();
125
126     }
127
128     //反转对焦状态, 若原来在持续对焦则暂停, 否则开启
129     if ( Key_Scan(KEY2_GPIO_PORT,KEY2_PIN) == KEY_ON ) {
130         if (focus_status == 1) {
131             //暂停对焦
132             OV5640_FOCUS_AD5820_Pause_Focus();
```

```
133         focus_status = 0 ;
134     } else {
135         //自动对焦
136         OV5640_FOCUS_AD5820_Constant_Focus();
137         focus_status = 1 ;
138     }
139 }
140
141
142
143 //使用串口输出帧率
144 if (Task_Delay[0]==0) {
145     /*输出帧率*/
146     CAMERA_DEBUG("\r\n 帧率: %.1f/s \r\n", (double) fps/5.0);
147     //重置
148     fps =0;
149
150     Task_Delay[0]=5000; //此值每 1ms 会减 1, 减到 0 才可以重新进来这里
151 }
152 }
153 }
154 }
155 }
```

阅读本代码时，先跳过前面的 Camera_Mode_Reconfig 函数。先来看在 main 函数，它初始化了液晶屏、串口、按键和定时器等外设。

摄像头控制部分，首先调用了 OV5640_HW_Init 函数初始化 DCMI 的 GPIO 及 I2C，然后调用 OV5640_ReadID 函数检测摄像头与实验板是否正常连接，若连接正常则调用 OV5640_Init 函数初始化 DCMI 的工作模式及 DMA，再调用 OV5640_RGB565_Default_Config 函数向 OV5640 写入默认的寄存器配置，调用 OV5640_USER_Config 把全局变量 cam_mode 的值写入摄像头，调用 OV5640_FOCUS_AD5820_Init 函数设置自动对焦，最后，一定要记得调用 ImagDisp 使能 DCMI 开始捕获数据，这样才能正常开始工作。

在 main 函数的 while 循环里，循环检测按键，若按下 key1 按键，它会调用开头部分的 Camera_Mode_Reconfig 函数设置新的 cam_mode 值，然后调用 OV5640_USER_Config 更新摄像头配置，它演示了如何在程序运行时利用 cam_mode 更新摄像头配置。

在 while 循环中，还有一个条件判断，它使用定时器每 5 秒统计一次帧率，并使用串口输出。

3. 下载验证

把 OV5640 接到实验板的摄像头接口中，用 USB 线连接开发板，编译程序下载到实验板，并上电复位，液晶屏会显示摄像头采集得的图像，摄像头会自动调焦。

联系我们

东莞野火电子有限公司

官网: www.embedfire.com

论坛: www.firebbs.cn

淘宝: fire-stm32.taobao.com

邮箱: firege@embedfire.com

电话: 0769-33894118



关注野火公众号，可下载野火全部产品的资料（书籍、视频、程序）



野火 STM32 系列图书，关注公众号即可下载电子版